

Archivos

Juan Arnedo Moreno

Programación básica (ASX)
Programación (DAM)
Programación (DAW)

Índice

Introducción	5
Resultados de aprendizaje	7
1 Gestión de archivos	9
1.1 La clase File	9
1.1.1 Inicialización	10
1.1.2 Rutas relativas y absolutas	11
1.1.3 Operaciones típicas de la clase File	14
1.2 Solución de los retos propuestos	22
2 Tratamiento básico de datos	25
2.1 Acceso secuencial a archivos orientados a carácter	26
2.1.1 Inicialización	26
2.1.2 Control de errores	28
2.1.3 Lectura de datos	31
2.1.4 Escritura de datos	34
2.2 Aspectos importantes del acceso secuencial	36
2.2.1 Detección de final de secuencia	37
2.2.2 Procesamiento de los datos leídos	39
2.2.3 Archivos con valores de diferentes tipos	44
2.3 Acceso secuencial a archivos orientados a byte	46
2.3.1 Inicialización	48
2.3.2 Escritura de datos	49
2.3.3 Lectura de datos	53
2.4 Acceso relativo a los datos	56
2.4.1 Posicionamiento	57
2.4.2 Lectura de datos relativa	60
2.4.3 Escritura de datos relativa	61
2.5 Soluciones a los retos propuestos	65
3 Tratamiento modular de datos. El juego de combates en la arena	71
3.1 Descripción del problema	71
3.1.1 División del problema	71
3.1.2 Módulos del programa	72
3.2 Criterios de elección de tipos de	73
archivo 3.3 La biblioteca "juego.arena.ficheros"	74
3.3.1 La clase Ranking	74
3.3.2 La clase Bestiario	77
3.3.3 La clase auxiliar EditorBestiari	82
3.4 Enmiendas a los módulos originales	85
3.4.1 En la clase EntradaTeclado	85
3.4.2 En la clase SalidaPantalla	86
3.4.3 En la clase Juego Arena	86

Introducción

La principal función de una aplicación informática es la manipulación y la transformación de datos. Estos datos pueden representar cosas muy diferentes según el contexto del programa: notas de estudiantes, una recopilación de temperaturas, las fechas de un calendario, etc. Las posibilidades son ilimitadas. Todas estas tareas de manipulación y transformación se llevan a cabo normalmente mediante el almacenamiento de los datos en variables, dentro de la memoria del ordenador, de modo que pueden aplicarse operaciones, ya sea mediante operadores o la invocación de métodos.

Desafortunadamente, todas estas variables sólo tienen vigencia mientras el programa se está ejecutando. Una vez finalizado el programa, los datos que contienen desaparecen. Esto no es problema para programas que siempre tratan los mismos datos, que pueden tomar la forma de literales en el programa. O cuando el número de datos a tratar es pequeño y se puede preguntar al usuario. Ahora bien, imagina tener que introducir las notas de todos los estudiantes cada vez que se ejecuta el programa para gestionarlas. No tiene sentido. Por tanto, en algunos casos, aparece la necesidad de poder registrar los datos en algún soporte de memoria externa, de forma que éstos se mantengan de forma persistente entre diferentes ejecuciones del programa, o incluso si se apaga el ordenador.

La forma más sencilla de alcanzar este objetivo es almacenar la información aprovechando el sistema de archivos que ofrece el sistema operativo. Mediante este mecanismo, es posible tener los datos en un formato fácil de gestionar e independiente del soporte real, ya sea un soporte magnético como un disco duro, una memoria de estado sólido, como un lápiz de memoria USB, un soporte óptico, cinta, etc.

En el apartado “Gestión de archivos” se presentan una serie de conceptos básicos vinculados a la gestión de archivos mediante el lenguaje Java. Sin querer enumerar cada una de las clases o métodos que conforman la biblioteca de entrada / salida de Java (java.io), se explica cómo llevar a cabo los primeros pasos para controlar y manipular el sistema de archivos del ordenador mediante código fuente en sus programas, independientemente de su sistema operativo.

Una vez se dispone de los conocimientos básicos para manipular el sistema de archivos, en el apartado “Tratamiento básico de datos” se explica cómo poder leer y grabar datos dentro de un archivo. Hay dos formas de llevar a cabo este proceso: mediante acceso secuencial, un sistema muy parecido a cómo se leerían por el teclado o se escribirían por pantalla, o relativo, que se aproxima más a cómo se gestionan los datos de un array. Independientemente del sistema de acceso usado, otro aspecto muy importante que tendrá que tener en cuenta y es necesario estudiar es el formato como almacenar los datos, como un archivo de texto o en su codificación binaria original.

Por último, el apartado “Tratamiento modular de datos” liga todos los conceptos vinculados a la gestión de archivos con los de diseño descendente y modularidad, usando como hilo argumental un ejemplo de programa de cierta complejidad. Se parte

de una aplicación ya existente y, aplicando el principio de modularidad, añadirá nuevas funcionalidades asociadas a la gestión de archivos.

Resultados de aprendizaje

Al finalizar esta unidad, el alumno/a:

1. Reconoce los conceptos relacionados con archivos.
2. Reconoce los distintos tipos de archivos.
3. Establece y diferencia las operaciones a realizar sobre los archivos en el lenguaje de programación empleado
4. Utiliza correctamente diferentes operaciones sobre archivos.
5. Modulariza adecuadamente los programas que gestionan archivos.
6. Diseña, prueba y documenta programas simples que gestionan archivos.

1. Gestión de archivos

Entre las funciones de un sistema operativo está la de ofrecer mecanismos genéricos para gestionar sistemas de archivos. Normalmente, dentro de un sistema operativo moderno (o ya no tan moderno), se espera disponer de algún tipo de interfaz o explorador para poder gestionar archivos, ya sea gráficamente o usando una línea de comandos de texto. Si bien la forma en que los datos se guardan realmente en los dispositivos físicos de almacenamiento de datos puede ser muy diferente según cada tipo (magnético, óptico, etc.), la forma de gestionar el sistema de archivos suele ser muy similar en la inmensa mayoría de los casos: una estructura jerárquica con carpetas y archivos.

Ahora bien, en realidad, la capacidad de operar con el sistema de archivos no es exclusiva de la interfaz ofrecida por el sistema operativo. Muchos lenguajes de programación proporcionan bibliotecas que permiten acceder directamente a los mecanismos internos que ofrece el sistema, por lo que es posible crear código fuente desde el que, con las instrucciones adecuadas, se pueden realizar operaciones típicas de un explorador de archivos. De hecho, las interfaces ofrecidas, como un explorador de archivos, son un programa como cualquier otro, el cual, usando precisamente estas librerías, permite que el usuario gestione archivos fácilmente. Pero no es extraño encontrar otras aplicaciones con su propia interfaz para gestionar archivos, aunque sólo sea para poder seleccionar cuál es necesario cargar o guardar en un momento dado: editores de texto, compresores, reproductores de música, etc.

Java no es ninguna excepción ofreciendo este tipo de biblioteca, en forma del conjunto de clases incluidas en el package `java.io`. Mediante la invocación de los métodos adecuados definidos de estas clases, es posible llevar a cabo prácticamente cualquier tarea sobre el sistema de archivos.

1.1 La clase File

La pieza más básica para poder operar con archivos, independientemente de su tipo, en un programa hecho en Java es el tipo compuesto `File`. Esta clase pertenece al package `java.io`. Por tanto, deberá importarlo antes de poder usarla. Ésta le permite manipular cualquier aspecto vinculado al sistema de archivos. Ahora bien, hay que tener cuidado, ya que su nombre (“fichero”, en inglés) es algo engañoso, ya que no se refiere exactamente a un archivo.

La clase `File` indica, más concretamente, una ruta dentro del sistema de archivos.

Concretamente, sirve para realizar operaciones tanto sobre rutas al sistema de archivos que ya existan como no existentes. Además, esta clase se puede utilizar tanto para manipular archivos de datos como carpetas.

1.1.1 Inicialización

Como cualquier otra clase, antes de poder trabajar con `File`, aparte de importarla correctamente, es necesario inicializarla, de forma que sea posible invocar sus métodos. En este caso, para ello es necesario incluir como parámetro una cadena de texto correspondiente a la ruta sobre la que se quieren realizar las operaciones.

```
1 File f = new File (String ruta)
```

Una ruta es la forma general de un nombre de archivo o carpeta, por lo que identifica únicamente su localización en el sistema de archivos.

Cada uno de los elementos de la ruta puede existir realmente o no, pero esto no impide en modo alguno poder inicializar `File`. En realidad, su comportamiento es como una declaración de intenciones sobre qué ruta del sistema de archivos se quiere interactuar. No es hasta que se llamen los diferentes métodos definidos en `File`, o hasta que se escriben o leen datos, que realmente se accede al sistema de archivos y se procesa la información.

Debe tenerse siempre presente si el sistema operativo distingue entre mayúsculas y minúsculas o no.

Un aspecto importante que debe tenerse presente al inicializar `File` es tener siempre presente que el formato de la cadena de texto que conforma la ruta puede ser diferente según el sistema operativo sobre el que se ejecuta la aplicación. Por ejemplo, el sistema operativo Windows inicia las rutas por un nombre de unidad (C:, D:, etc.), mientras que los sistemas operativos basados en Unix comienzan directamente con una barra ("/"). Además, los diferentes sistemas operativos usan diferentes separadores en las rutas. Por ejemplo, los sistemas Unix usan la barra ("/") mientras que Windows la contrabarra ("\\").

- Ruta Unix: `/usr/bin`
- Ruta Windows: `C:\Windows\System32`

Para generar aplicaciones totalmente portables en diferentes sistemas, la clase `File` ofrece la posibilidad de acceder a una constante declarada dentro de esta clase para especificar separadores de ruta en una cadena de texto de forma genérica, llamada `File.separator`. Cuando se usa, su valor es el que corresponde según el sistema operativo en el que se ejecuta el programa. El acceso a esta constante se hace de forma muy parecida a cómo se invoca un método estático.

```
1 File f = new File(File.separator + "usr" + File.separator + "bin");
```

De todas formas, Java adopta por defecto el sistema Unix como separador por defecto (usando la barra, /), independientemente del sistema operativo. Por tanto, en un sistema Windows es posible separar una ruta con este carácter y Java lo sabrá interpretar correctamente. A partir de ahora, todos ejemplos se basarán en un sistema Windows (rutas iniciadas con un nombre de unidad, C:, D:, etc.), pero se utilizará la barra para referirse a cualquier ruta.

Para escribir una contrabarra en una cadena de texto es necesario escribir '\\.

De hecho, al hacer un programa Java en un sistema Windows hay que ser especialmente cuidadoso con este hecho, ya que la contrabarra no es un carácter permitido dentro de una cadena de texto, y sirve para declarar valores especiales de escape (\n salto de línea, \t tabulador, etc.).

Otro aspecto muy importante al inicializar File es que se trata de una clase que define un tipo compuesto (como String), y no un simple repositorio de métodos (como Scanner o Random). En consecuencia, cada vez que se inicializa, al identificador empleado se le asigna un valor asociado a esa ruta. A partir de entonces, el identificador actúa como una variable en el programa. Para operar con diferentes rutas al mismo tiempo será necesario inicializar y manipular diferentes variables, al igual que se haría con distintas cadenas de texto.

Por ejemplo, en el siguiente código se declaran tres variables diferentes, cada una de las cuales almacena un valor de tipo File diferente. Para trabajar con cada ruta, ya sea carpeta o archivo, será necesario usar la variable correspondiente.

```
1 ...  
2 File carpetaFotos = new File("C:/Personal/MisFotos"); 3 File unaFoto = new File("C:/Personal/MisFotos/Foto1.png"); 4  
File otraFoto = new File("C:/Personal/MisFotos/Foto2.png");  
5 ...
```

1.1.2 Rutas relativas y absolutas

En los ejemplos utilizados hasta el momento para inicializar variables del tipo File se han usado rutas absolutas, ya que es la forma de dejar más claro a qué elemento dentro del sistema de archivos, ya sea archivo o carpeta, se está haciendo referencia.

Una ruta absoluta es aquella que se refiere a un elemento destino a partir de la carpeta raíz de un sistema de archivos. En ella se enumeran, una por una, todas las carpetas que hay entre la carpeta raíz hasta la carpeta que contiene el elemento destino.

Las rutas absolutas se distinguen fácilmente, ya que el texto que las representa comienza de una forma muy característica dependiendo del sistema operativo del ordenador. En el caso de los sistemas operativos Windows en su inicio siempre se pone el nombre de la unidad ("C:", "D:", etc.), mientras que en el caso de los sistemas operativos Unix, éstas comienzan siempre por una barra ("/").

Por ejemplo, las siguientes cadenas de texto representan rutas absolutas dentro de un sistema de archivos de Windows:

- C:\Fotos\Viajes (ruta a una carpeta)
- M:\Documents\Unidad6\Apartado1 (ruta a una carpeta)
- N:\Documents\Unidad6\Apartado1\Actividades.txt (ruta a un archivo)

En cambio, en el caso de una jerarquía de archivos bajo un sistema operativo Unix, un conjunto de rutas podrían estar representadas de la siguiente forma:

- /Fotos/Viajes (ruta a una carpeta)
- /Documentos/Unidad6/Apartado1 (ruta a una carpeta)
- /Documents/Unidad6/Apartado1/Actividades.txt (ruta a un archivo)

Al inicializar variables de tipo File usando una ruta absoluta, siempre es necesario usar la representación correcta según el sistema en el que se ejecuta el programa.

Si bien el uso de rutas absolutas resulta útil para indicar con toda claridad qué elemento dentro del sistema de archivos se está manipulando, hay casos en que su uso comporta ciertas complicaciones. Suponga que ha realizado un programa en el que se realizan operaciones sobre el sistema de archivos. Una vez que ve que funciona, deje el proyecto Java a un amigo, que lo copia en su ordenador dentro de una carpeta cualquiera y lo abre con su entorno de trabajo. Para que el programa le funcione perfectamente, antes será necesario que en su ordenador haya exactamente las mismas carpetas que ha usado en su ordenador, tal y como están escritas en el código fuente de su programa. De lo contrario, no funcionará, ya que las carpetas y archivos esperados no existirán, y por tanto, no se encontrarán. Usar rutas absolutas hace que un programa siempre tenga que trabajar con una estructura del sistema de archivos exactamente igual esté donde esté donde se ejecute, cosa no muy cómoda.

Para resolver este problema, a la hora de inicializar una variable tipo File, también se puede hacer refiriéndose a una ruta relativa.

En los sistemas Windows, dentro de las propiedades de un atajo, existe una opción para especificar el "Directorio de trabajo" de su aplicación asociada.

Una ruta relativa es aquella que se considera que parte de la carpeta, o directorio, de trabajo de la aplicación. Esta carpeta puede ser diferente cada vez que se ejecuta el programa y depende de cómo se ha llevado a cabo esta ejecución.

Cuando se ejecuta un programa, ya sea en Java o en cualquier otro lenguaje de programación, por defecto se le asigna una carpeta de trabajo. Esta carpeta suele ser, por defecto, la carpeta desde la que se lanza el programa, si bien existen sistemas operativos que permiten especificarla explícitamente mediante alguna opción sobre sus atajos o archivos ejecutables. En el caso de un programa en Java ejecutado a través de un IDE, la carpeta de trabajo suele ser la misma carpeta en la que se ha elegido guardar los archivos del proyecto.

El formato de una ruta relativa es similar a una ruta absoluta, pero nunca se indica la raíz del sistema de archivos. Directamente se empieza por el primer elemento escogido en la ruta. Por ejemplo:

- Viajes
- Unidad6\Apartado1
- Unidad6\Apartado1\Actividades.txt

La propiedad especial de una ruta relativa es que, implícitamente, se considera que ésta siempre incluye el directorio de trabajo de la aplicación como parte inicial del texto, aunque no se haya escrito. Ahora bien, el rasgo distintivo es que, precisamente, este directorio de trabajo puede variar. Por ejemplo, dada la siguiente inicialización, el elemento al que se refiere la ruta varía según indica la tabla 1.1:

```
1 File f = new File ("Unidad6/Apartado1/Actividades.txt");
```

Tabla 1. 1. Rangos y palabras clave de los tipos primitivos numéricos en Java

Directorio de trabajo	Ruta real
C:\Proyectos\Java	C:\Proyectos\Java\Unidad6\Apartado1\Actividades.txt
D:\Unidades	X:\Unidades\Unidad6\Apartado1\Actividades.txt
/Programas	/Programas/Unidad6/Apartado1/Actividades.txt

Este mecanismo permite minimizar el número de carpetas que hay que garantizar que existen cuando ejecutamos programas en diferentes equipos, ya que sólo es necesario controlar la vinculada a la ruta relativa. La parte vinculada al directorio de trabajo varía automáticamente por cada versión copiada en cada máquina. En el caso más sencillo de entender, suponga la siguiente inicialización:

```
1 File f = new File ("Actividades.txt");
```

Dada esta ruta, basta con garantizar que el archivo "Activitats.txt" está siempre en el mismo directorio de trabajo de la aplicación, sea cual sea éste (en un ordenador puede ser "C:\Programes", en otro "/Java", etc.). En cualquiera de estos casos, la ruta siempre será correcta. De hecho, aún más. Nótese cómo las rutas relativas a Java permiten crear código independiente del sistema operativo, ya que no es necesario especificar un formato de raíz ligada a un sistema de archivos concreto ("C:", "D:", "/", etc.).

Java dispone de dos instrucciones para consultar cuál es la carpeta de trabajo de un programa y cambiarla por otro, si así se desea. De todas formas, normalmente se deja la carpeta que asigna por defecto el sistema operativo al ejecutar el programa.

```
1 //Asigna en forma de cadena de texto la ruta de la carpeta de trabajo actual.  
2 String dirTrebll = System.getProperty("user.dir");
```

```
1 //Indica cuál es la ruta de la nueva carpeta de trabajo a partir de una cadena  
  de texto.  
2 String nuevoDirTrebll = "...";  
System.setProperty("user.dir", nuevoDirTrabajo);
```

De todas formas, a medida que vaya cómo realizar operaciones con archivos a partir de rutas, se verá más claro el funcionamiento de rutas relativas y absolutas.

1.1.3 Operaciones típicas de la clase File

Al contrario que las cadenas de texto, el tipo compuesto File no permite usar absolutamente ningún tipo de operador, ni siquiera el '+'. Cualquier operación que se quiera llevar a cabo con una ruta deberá realizarse mediante la invocación de métodos sobre la variable correspondiente. En este sentido, File ofrece varios métodos para poder manipular el sistema de archivos u obtener información a partir de su ruta. Algunos de los más significativos para entender sus funcionalidades se muestran a continuación, ordenados por tipos de operación.

Esta vez, para describir cada método se utilizará su definición tal y como aparece en la documentación de Java, especificando el tipo del valor de retorno y de sus parámetros.

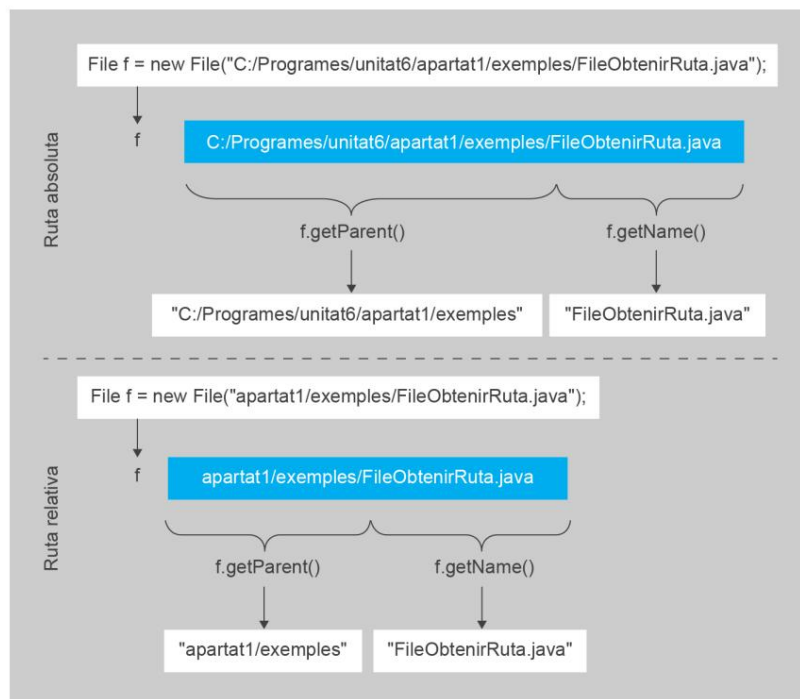
Obtención de la ruta

Una vez inicializada una variable de tipo File, puede ser necesario recuperar la información utilizada durante su inicialización y conocer en formato texto a qué ruta se está refiriendo, o como mínimo parte de ella.

- `String getParent()` evalúa la ruta de la carpeta padre del elemento referido por esta ruta. Básicamente, la cadena de texto resultante es idéntica a la ruta original utilizada al inicializar la variable de tipo File, eliminando el último elemento. Si la ruta tratada se refiere a la carpeta raíz de un sistema de archivos ("C:\", "/", etc.), este método evalúa null. En caso de tratarse de una ruta relativa, este método no incluye la parte de la carpeta de trabajo.
- `String getName()` evalúa El nombre de El elemento que representa esta ruta, ya sea una carpeta o un archivo. Se trata del caso inverso del método `getParent()`, puesto que el texto resultante es sólo el último elemento, descartando el resto del texto de la ruta previa.
- `String getAbsolutePath()` evalúa el texto relativo a la ruta absoluta. Si la variable File se inicializó usando una ruta relativa, el resultado incluye también la parte asociada a la carpeta de trabajo.

La figura 1.1 muestra un resumen del texto que evalúa cada método, dada una ruta concreta.

Figura 1. 1. Evaluación de los métodos de obtención de ruta en File



El siguiente programa muestra un ejemplo de cómo funcionan estos tres métodos.

Nótese cómo a las rutas relativas se les añade la carpeta de trabajo, que será la carpeta donde tiene su proyecto con el código fuente del programa.

```

1 package unidad6.apartado1.ejemplos;
2 //Hay que importar la clase File
3 import java.io.File;
4 public class FileObtenerRuta {
5     public static void main(String[] args) {
6         FileObtenerRuta programa = new FileObtenerRuta();
7         programa.inicio();
8     }
9     public void inicio() {
10        //Se inicializan dos rutas absolutas diferentes
11        File carpetaAbs= new File ("c:/Temp");
12        File archivoAbs = new File ("c:/Temp/Document.txt");
13        //Y unas rutas relativas
14        File carpetaRel = new File ("Tiempo");
15        File archivoRel = new File ("Tiempo/Documento.txt");
16        //Mostramos los datos de cada una
17        mostrarRutas(carpetaAbs);
18        mostrarRutas(archivoAbs );
19        mostrarRutas(carpetaRel );
20        mostrarRutas(archivoRel );
21    }
22    public void mostrarRutas(File f) {
23        System.out.println("La ruta es " + f.getAbsolutePath());
24        System.out.println("Su padre es " + f.getParent());
25        System.out.println("Su nombre es " + f.getName() + "\n");
26    }
27 }

```

Comprobaciones de estado

Dada la ruta utilizada para inicializar una variable de tipo File, ésta puede ser que realmente exista dentro del sistema de archivos o no, ya sea en forma de

archivo o carpeta. La clase File ofrece un conjunto de métodos que permiten realizar comprobaciones sobre su estado y saber si es así.

boolean exists() comprueba si la ruta existe dentro del sistema de archivos o no. Evaluará true o false, para cada caso, respectivamente. Se considera que la ruta no existe si cualquiera de los elementos individuales que la conforman no existe.

Normalmente, dado el texto de una ruta, los archivos se distinguen de las carpetas por su formato, al tener extensión (el último elemento de la ruta suele ser de la forma nombre.extensión, como Foto1.jpg o Document.txt). Sin embargo, hay que tener en cuenta que la extensión no es un elemento obligatorio en el nombre de un archivo, sólo se usa como a mecanismo para que tanto el usuario como algunos programas puedan discriminar más fácilmente tipos de archivos. Por tanto, sólo con el texto de una ruta no se puede estar 100% seguro de si ésta se refiere a un archivo o a una carpeta. Para poder estar realmente seguros, se pueden usar los siguientes métodos:

- boolean isFile() permite comprobar si la ruta se refiere a un archivo (evaluando true) o no (evaluando false). Esta comprobación no se lleva a cabo únicamente sobre el texto de la ruta, sino que se mira en el sistema de archivos si la ruta existe, y entonces se mira si el elemento con este nombre se trata de un archivo o carpeta. Si no existe, siempre se evalúa a false.
- boolean isDirectory() funciona igual que el anterior, pero comprobando si se trata de una carpeta.

Por ejemplo, el siguiente programa realiza una serie de comprobaciones sobre un conjunto de rutas. Pero para poder probarlo de manera efectiva, habría que preparar el sistema de archivos de su ordenador. Por ejemplo, puede crear una carpeta llamada "Tiempo" en la raíz de la unidad "C:". Dentro de ésta, crea otra llamada "Fotos". En la misma carpeta "Temp", cree un nuevo archivo (que puede estar vacío) llamado "Document.txt". Luego, borre la carpeta o el documento y vuelva a ejecutar el programa.

```
1 package unidad6.apartado1.ejemplos;
2 importe java.io.File;
3 public class FileComprobarArchivos {
4     public static void main(String[] args) {
5         FileComprovarArchivos programa = new FileComprovarArchivos();
6         programa.inicio();
7     }
8     public void inicio() {
9         File temp = new File ("C:/Temp");
10        File fotos = new File ("C:/Tiempo/Fotos");
11        File document = new File ("C:/Temp/Document.txt");
12        System.out.println(temp.getAbsolutePath() + mostrarEstado(fotos);        " existe? "        + temp.exists());
13
14        mostrarEstado(documento);
15    }
16    public void mostrarEstado(File f) {
17        System.out.println(f.getAbsolutePath() +        " es un archivo? " + f.isFile());
18        System.out.println(f.getAbsolutePath() + isDirectory());        " es una carpeta? " + f.
19    }
20 }
```

Lectura y modificación de propiedades

Cuando la ruta con la que se trabaja corresponde a un archivo o carpeta que realmente existe dentro del sistema, hay toda una serie de información suministrada por el sistema operativo que puede resultar útil conocer: sus atributos de acceso, su tamaño, fecha de modificación, etc. Por lo general, todos los datos mostrados al acceder en las propiedades del archivo. Esta información también puede ser consultada usando los métodos adecuados. Entre los más populares se encuentran los siguientes:

- `long length()` evalúa el tamaño de un archivo, en bytes. Este método sólo puede ser invocado sobre una ruta que represente un archivo, de lo contrario no se puede garantizar la corrección del resultado (no se puede considerar como válido en ningún caso).
- `long lastModified()` evalúa la última fecha de edición del elemento representado por esta ruta. El resultado se codifica en un único número entero donde se mide el número de milisegundos que han pasado desde el 1 de junio de 1970.

De milisegundos a una fecha legible

Lo cierto es que, para mostrar datos al usuario, trabajar con milisegundos para indicar una fecha no es muy cómoda. Afortunadamente, en el package `java.util` es posible encontrar un tipo compuesto auxiliar que permite trabajar con fechas (al igual que `File` permite trabajar con rutas de archivos). Se trata de la clase `Date`. Para inicializarla, precisamente, es necesario usar como parámetro el número de milisegundos desde el 1 de junio de 1970, en formato `long`.

```
Date data = new Date(1306751122651L);
```

No se entrará en mayor detalle sobre cómo funciona esta clase. Si lo desea, puede mirar los sus métodos en la documentación de la API de Java.

El siguiente ejemplo muestra cómo funcionan estos métodos. Para probarlos, antes cree un archivo llamado "Document.txt" en la carpeta "C:\Temp". Inicialmente, déjelo vacío y ejecute el programa. Luego, con un editor de texto, escriba cualquier cosa, guarde los cambios y vuelva a ejecutar el programa. Observe como el resultado es distinto. Fíjese también en el uso de la clase `Date` para poder mostrar correctamente la fecha en un formato legible.

```
1 package unidad6.apartado1.ejemplos;
2 importe java.io.File;
3 importe java.util.Date;
4 public class FileLeerPropiedades {
5     public static void main(String[] args) {
6         FileLlegirPropiedades programa = new FileLlegirPropiedades();
7         programa.inicio();
8     }
9     public void inicio() {
10        File document = new File("C:/Temp/Document.txt");
11        System.out.println(document.getAbsolutePath());
12        //Se usa el tipo compuesta Date para transformar milisegundos a fecha real
13        Date data = new Date(document.lastModified());
14        System.out.println("Última modificación: " + fecha);
15        System.out.println("Tamaño: " + document.length());
16    }
17 }
```

Gestión de archivos

Seguro, el conjunto de operaciones más habituales cuando accede a un sistema de archivos al trabajar en su ordenador son las vinculadas a su gestión directa: renombrar archivos, borrarlos, copiarlos o moverlos. Dado el nombre de una ruta, Java también permite realizar estas acciones.

- `boolean mkdir()` permite crear una carpeta de acuerdo a la ubicación que indica la ruta. La ruta debe indicar el nombre de una carpeta que no existe en el momento de invocar el método. Concretamente, se crea una carpeta con el nombre de la ruta (el último elemento del texto), ubicada en la carpeta padre indicada (todo el texto anterior). Si se ha creado correctamente, el método evalúa `true`. En de lo contrario, evalúa `false`. Hay varios motivos por los que puede fallar la creación, pero normalmente se debe a que la ruta es incorrecta (alguno de los elementos no existen) o la carpeta ya existía.
- `boolean delete()` borra el archivo o carpeta que indica la ruta. La ruta debe indicar el nombre de una carpeta que sí existe en el momento de invocar el método. Al igual que en el anterior método, evalúa `true` o `false` según si la operación realmente se ha podido realizar. Los motivos por los que puede fallar son similares a lo anterior, si bien en el caso de las carpetas también puede fallar si no está vacía. Sólo se puede borrar una carpeta usando este método si dentro su no tiene nada (ni otras carpetas ni archivos).

Para probar el ejemplo que se muestra a continuación para que se pueda ver cómo funcionan estos métodos, nuevamente, es necesaria una organización previa del sistema de archivos. Primero, asegúrese de que en la raíz de la unidad "C:" no hay ninguna carpeta llamada "Temp" y ejecute el programa. Todo fallará, ya que las rutas son incorrectas (no existe "Temp"). Una vez hecho, cree la carpeta "Temp", y en su interior crea un nuevo documento llamado "Document.txt" (que puede estar vacío si desea). Ahora, al ejecutar el programa, se habrá creado una nueva carpeta llamada "Fotos". Si sin tocar la carpeta "Temp" ejecute el programa por tercera vez, ahora todo se habrá borrado.

```
1 package unidad6.apartado1.ejemplos;
2 importe java.io.File;
3 public class FileGestioElements {
4     public static void main(String[] args) {
5         FileGestioElements programa = new FileGestioElements();
6         programa.inicio();
7     }
8     public void inicio() {
9         File fotos = new File ("C:/Tiempo/Fotos");
10        File document = new File ("C:/Temp/Document.txt");
11        boolean resultado= fotos.mkdir();
12        System.out.println("Creada carpeta if (!resultado) " + fotos.getName() + "? " + resultado);
13        {
14            boolean de Carpeta = fotos.delete();
15            System.out.println("Borrada carpeta de la Carpeta); " + fotos.getName() + "? " +
16
17            boolean de Archivo= document.delete();
18            System.out.println("Borrado archivo" de Archivo); " + document.getName() + "? " +
19
20        }
```

Desde el punto de vista de un sistema operativo, la operación de “mover” un archivo o carpeta no es más que cambiar su nombre, desde su ruta original hasta una nueva ruta destino. Para ello también hay un método, ya algo más complejo boolean `renameTo(File destino)`.

boolean `renameTo(File desti)` es un método con un nombre algo engañoso (“renombrar”, en inglés), ya que su función real no es simplemente cambiar el nombre de un archivo o carpeta, sino cambiar la ubicación completa. El método se invoca sobre la variable con la ruta que se considera el archivo origen, y el parámetro de entrada es la ruta destino. En este caso, el archivo o carpeta asociado a la ruta origen debe existir en el momento de invocar el método, pero el destino no. Como en casos anteriores, evalúa true o false según si la tarea se ha podido llevar a término correctamente o no (la ruta origen y destino son correctos, no existe ya un archivo con ese nombre en el destino, etc.). Nótese que, en el caso de carpetas, es posible moverlas aunque contengan archivos. Toda la estructura jerárquica de carpetas se desplaza.

Una vez más, es necesaria una cierta preparación previa en la ejecución del ejemplo que permite ver cómo funciona con más detalle este método. Dentro de una carpeta llamada “Temp” en la unidad “C:”, cree una carpeta llamada “Media” y una otra llamada “Fotos”. Dentro de la carpeta “Fotos”, genere dos documentos (que pueden estar vacíos), llamados “Document.txt” y “Fotos.txt”. Después de ejecutar el programa, observe cómo la carpeta “Fotos” se ha movido y ha cambiado de nombre, pero mantiene en su interior el archivo “Fotos.txt”. El archivo “Document.txt” se ha movido hasta la carpeta “Temp”.

```
1 package unidad6.apartado1.ejemplos;
2 importe java.io.File;
3 public class FileMoure {
4     public static void main(String[] args) {
5         FileMoure programa = new FileMoure();
6         programa.inicio();
7     }
8     public void inicio() {
9         File origenCarpeta = new File("C:/Temp/Fotos");
10        File destiCarpeta = new File("C:/Temp/Media/Fotografías");
11        File origenDocument = new File("C:/Temp/Media/Fotografias/Document.txt");
12        File destiDocument = new File("C:/Temp/Document.txt");
13        boolean resultado = origenCarpeta.renameTo(destiCarpeta);
14        System.out.println("¿Se ha movido y renombrado la carpeta? " + resultado);
15        origenDocument.renameTo(destiDocument);
16        System.out.println("¿Se ha movido el documento? " + resultado);
17    }
18 }
```

Como puede verse en el ejemplo, este método también sirve, implícitamente, para renombrar archivos o carpetas. Si el elemento final de las rutas origen y destino son diferentes, el nombre del elemento, sea archivo o carpeta, cambiará. Por simplemente renombrar un elemento sin moverlo de sitio, basta con que su ruta padre sea exactamente la misma. El resultado es que el elemento de la ruta origen “se mueve” a la misma carpeta en la que está ahora, pero con un nombre diferente.

Por ejemplo, el código siguiente renombra un archivo llamado “Document.txt”, ubicado en la carpeta “C:/Temp”, en “Renombrado.txt”:

```
1 package unidad6.apartado1.ejemplos;  
2 importe java.io.File;  
3 public class FileRenombrar {  
4     public static void main(String[] args) {  
5         FileRenombrar programa = new FileRenombrar();  
6         programa.inicio();  
7     }  
8     public void inicio() {  
9         File rutaOrigen = new File("C:/Temp/Document.txt");  
10        //Renombrar: misma ruta padre pero distinto nombre final  
11        String nuevoNombre = rutaOrigen.getParent() + File.separator + "Renombrado.txt";  
12        File rutaDesti = new File(nuevoNombre);  
13        rutaOrigen.renameTo(rutaDesti);  
14    }  
15 }
```

La creación de archivos por el momento se dejará pendiente, ya que está vinculada al tratamiento de los datos que debe contener, y por tanto va más allá de la mera gestión de una ruta.

Reto 1. Cree un programa que lea un texto por el teclado correspondiente a la ruta de un archivo existente en su ordenador. Cambie el nombre de este archivo de modo que se elimine su extensión, si la tiene (por ejemplo, si el archivo se llama "Document.txt", pasará a llamarse "Documento"). Para llevar a cabo esta tarea tendrá que hacer memoria de las transformaciones sobre cadenas de texto.

Listado

Por último, sólo en el caso de las carpetas, es posible consultar cuál es el listado de archivos y carpetas que contiene.

`File[] listFiles()` evalúa un array de `File` donde se enumeran todos los elementos contenidos en la carpeta indicada por el texto usado al inicializar la variable de tipo `File`. Para que se ejecute correctamente, el texto de la ruta debe indicar forzosamente una carpeta. En cada posición de este array se encuentra el nombre de la ruta de cada uno de los archivos. El tamaño del array será igual al número de elementos que contiene la carpeta. Si el tamaño es 0, el valor devuelto es null y toda operación posterior sobre la variable resultante será errónea. El orden de los elementos puede ser cualquiera, no se puede garantizar nada (al contrario que en el explorador de archivos del sistema operativo, no se ordena automáticamente primero carpetas y después archivos, o alfabéticamente).

La particularidad que quizás hace un poco complejo este método, a primera vista, es el hecho de que evalúe un array de `File`. Dado que la clase `File` es un tipo compuesto, de cara a trabajar con él sólo tiene que recordar cómo se trabaja con un array de cadenas de texto, puesto que es lo mismo desde el punto de vista de acceso a las posiciones e invocación de métodos sobre los valores almacenados.

Pero lo mejor es ver un ejemplo. Antes de ejecutarlo, cree una carpeta llamada "Temp" en la raíz de la unidad "C:". Dentro de él crea o copia cualquiera cantidad de carpetas o archivos.

```
1 package unidad6.apartado1.ejemplos;
2 importe java.io.File;
3 public class FileLlistarElements {
4     public static void main(String[] args) {
5         FileLlistarElements programa = new FileLlistarElements();
6         programa.inicio();
7     }
8     public void inicio() {
9         File carpeta = new File("C:/Temp");
10        File[] arrayElements = carpeta.listFiles();
11        System.out.println("El contenido de " + carpeta.getAbsolutePath() + " es:")
            ;
12        //Para recorrer un array, se usa un bucle
13        for (int i = 0; i < arrayElements.length; i++) {
14            File f = arrayElements[i];
15            if (f.isDirectory()) {
16                System.out.print("[DIR] ");
17            } else {
18                System.out.print("[FIT] ");
19            }
20            System.out.println(f.getName());
21        }
22    }
23 }
```

Reto 2. Realice un programa que lea desde el teclado el texto asociado a la ruta a una carpeta existente dentro de su ordenador. Borrar todos los elementos que hay ha dentro, independientemente de que sean archivos o carpetas. Tenga en cuenta que para poder eliminar una carpeta es necesario vaciarla de todo su contenido (en la que puede haber al mismo tiempo tanto archivos o carpetas... y así indefinidamente hasta llegar a una carpeta que sólo tiene archivos). Pista: Para resolver este problema es mucho más sencillo plantear una solución recursiva.

1.2 Solución de los retos propuestos

Reto 1

```
1 package unidad6.apartado1.retos;
2 importe java.io.File;
3 importe java.util.Scanner;
4 public class CambiarNombre {
5     public static void main(String[] args) {
6         CambiarNombre programa = new CambiarNombre();
7         programa.inicio();
8     }
9     public void inicio() {
10         File rutaArchivo = leerNombreArchivo();
11         //Hay que comprobar si realmente existe y si es un archivo
12         if (rutaArchivo.isFile()) {
13             File nuevaRuta = quitarExtension(rutaArchivo);
14             //Se cambia el nombre
15             rutaArchivo.renameTo(nuevaRuta);
16             System.out.println("Nombre cambiado de " + rutaArchivo + " a " + nuevaRuta);
17
18             System.out.println("¡Este archivo no existe!");
19         }
20     }
21     /** Pregunta al usuario el nombre del archivo, y de éste obtiene una ruta.
22      *
23      * @return La ruta asociada al texto que ha escrito el usuario.
24      */
25     public File leerNomArchivo() {
26         Scanner lector = new Scanner(System.in);
27         System.out.println("Escribe el nombre de una ruta en un archivo existente: ");
28         String nombreArchivo = lector.nextLine();
29         //Fijaos cómo es posible hacer return de una variable de tipo File,
30         //igual que se haría por un entero o una cadena de texto.
31         File f = new File(nombreArchivo);
32         return f;
33     }
34     /** Dada una ruta, crea una nueva igual, pero sin extensión (.xxx)
35      *
36      * @param original Ruta original a transformar
37      * @return La ruta sin extensión
38      */
39     public File quitarExtension(File original) {
40         String nombre = original.getName();
41         String padre = original.getParent();
42         //Buscamos el último punto, para encontrar la extensión
43         int posicioPunt = nombre.lastIndexOf(".");
44         if (posiciónPunto >= 0) {
45             //eliminamos lo que hay detrás del punto
46             String nuevoNombre = nombre.substring(0, posicioPunto);
47             //Se hace el texto para la nueva ruta, sin extensión
48             String nuevoText = padre + File.separator + nuevoNombre;
49             File nuevaRuta = new File(nuevoText);
50             return nuevaRuta;
51         } else {
52             //Si no tiene extensión, se deja todo igual...
53             return original;
54         }
55     }
56 }
```

Reto 2

```
1 package unidad6.apartado1.retos;
2 importe java.io.File;
3 importe java.util.Scanner;
4 public class LimpiarCarpeta {
5     public static void main(String[] args) {
6         LimpiarCarpeta programa = new LimpiarCarpeta();
7         programa.inicio();
8     }
9     public void inicio() {
10         File rutaCarpeta = leerRutaCarpeta();
11         limpiarCarpeta(rutaCarpeta);
12     }
13     /** Pregunta al usuario el nombre de la carpeta, y de éste obtiene una ruta.
14      *
15      * @return La ruta asociada al texto que ha escrito el usuario.
16      */
17     public File leerRutaCarpeta() {
18         //Un tipo compuesto se puede dejar sin inicializar poniendo "null"
19         File f = null;
20         boolean preguntar = true;
21         Scanner lector = new Scanner(System.in);
22         while (preguntar) {
23             System.out.println("Escribe el nombre de una ruta en una carpeta: ");
24             String nombreCarpeta = lector.nextLine();
25             f = new File(nombreCarpeta);
26             if (f.isDirectory()) {
27                 preguntar = false;
28             } else {
29                 System.out.println("Esta carpeta no existe...");
30             }
31         }
32         return f;
33     }
34     /** Dada una ruta asociada a una carpeta, borra todo su contenido.
35      *
36      * @param ruta Ruta de la carpeta a limpiar
37      */
38     public void limpiarCarpeta(File ruta) {
39         File[] elementos = ruta.listFiles();
40         //Hay que mirar todos los elementos:
41         for (int i = 0; i < elementos.length; i++) {
42             if (elementos[i].isFile()) {
43                 //Si es un archivo, se borra.
44                 elementos[i].delete();
45             } else if (elementos[i].isDirectory()) {
46                 //Si es una carpeta, hay que vaciarla primero, o sea: invocar
47                 //limpiarCarpeta!
48                 limpiarCarpeta(elementos[i]);
49                 //Una vez vacía, se puede borrar correctamente
50                 elementos[i].delete();
51             }
52         }
53 }
```


2. Tratamiento básico de datos

Normalmente, las aplicaciones que utilizan archivos no están centradas en la gestión del sistema de archivos de su ordenador. El objetivo principal de usar archivos es poder almacenar datos, de modo que entre diferentes ejecuciones del programa, incluso en distintos equipos, es posible recuperarlos. El caso más típico es un editor de documentos, que mientras se ejecuta se encarga de gestionar los datos relativos al texto que está escribiendo, pero en cualquier momento puede guardarlo en un archivo para poder recuperar este texto en cualquier momento posterior, y añadir de nuevo, si procede. El archivo con los datos del documento lo puede abrir tanto en el editor de su ordenador como en el de otro compañero.

Para saber cómo tratar los datos de un archivo dentro de un programa, es necesario tener muy claro cómo se estructuran en él. Dentro de un archivo se pueden almacenar todo tipo de valores de cualquier tipo de datos. La parte más importante es que estos valores se almacenan en forma de secuencia, uno tras otro. Por tanto, como pronto verá, la manera más habitual de tratar archivos es secuencialmente, de manera parecida a cómo se hace para leerlas del teclado, mostrarlas por pantalla o recorrer las posiciones de un array.

Se llama acceso secuencial al tratamiento de un conjunto de elementos de modo que sólo es posible acceder a ellos de acuerdo a su orden de aparición.

Para poder tratar un elemento, es necesario haber tratado todos los elementos anteriores.



Un ejemplo de acceso secuencial a datos:
el mensaje de un telégrafo.
Fuente: Wallace Study-Telegraph

Java, junto con otros lenguajes de programación, diferencia entre dos tipos de archivos según se representen los valores almacenados dentro de un archivo.

En los archivos orientados a carácter, los datos se representan como una secuencia de cadenas de texto, donde cada valor se diferencia del otro usando un delimitador. En cambio, en los archivos orientados a byte, los datos se representan directamente de acuerdo a su formato en binario, sin separación alguna.

Si bien, a grandes rasgos, el acceso secuencial a los datos sigue el mismo esquema independientemente del tipo de archivo, cada caso tiene ciertas particularidades que hace interesante estudiarlos de forma diferenciada.

2.1 Acceso secuencial a archivos orientados a carácter

Un archivo orientado a carácter no es más que un documento de texto, como el que puede generar con cualquier editor de texto simple. Los valores están almacenados según su representación en cadena de texto, exactamente en el mismo formato que ha usado hasta ahora para entrar datos desde el teclado. Del mismo modo, los distintos valores se distinguen al estar separados entre sí con un delimitador, que por defecto es cualquier conjunto de espacios en blanco o salto de línea. Aunque estos valores se puedan distribuir en líneas de texto diferentes, conceptualmente, se puede considerar que están organizados uno tras otro, secuencialmente, como las palabras en la página de un libro.

El siguiente podría ser el contenido de un archivo orientado a texto donde hay diez valores de tipo real:

```
1 1,5 0,75 -2,35 18 9,4 3,14 16 -15,785 2 -200,4
2,56 9,3785
```

Y el de un archivo con 4 valores de tipo String ("Hi", "había", "una" y "vegada..."):

```
1 Érase una vez...
```

La principal ventaja de un fichero de este tipo es, pues, que resulta muy sencillo inspeccionar su contenido, generarlos de acuerdo a sus necesidades o representar cuál debe ser su contenido dentro del texto de otro documento, como acaba de hacerse.

2.1.1 Inicialización

Para el caso de los archivos orientados a carácter, es necesario usar dos clases diferentes según si lo que se desea es leer o escribir datos en un archivo. Normalmente, esto no es demasiado problemático, ya que en un bloque de código dado sólo se llevarán a cabo operaciones de lectura de datos o de escritura sobre un mismo archivo, pero no los dos tipos de operación a la vez. Normalmente, se leen los datos al inicio del programa, al igual que se puede pedir escribirlos usando el teclado, y se guardan cuando se dispone de los resultados a calcular, como cuando correspondería mostrarlos por pantalla.

Para tratar de forma sencilla archivos orientados a carácter, Java ofrece las clases `Scanner`, perteneciente al package `java.util`, y `PrintStream`, perteneciente al package `java.io`.

Como en el caso de cualquier otra clase que necesite usar dentro de su programa, antes de poder realizar ninguna operación es imprescindible inicializarla, de forma que se disponga de una variable sobre la que se pueden invocar los métodos correspondientes.

Scanner

La clase que permite llevar a cabo la lectura de datos desde un archivo orientado a carácter es exactamente la misma que permite leer datos desde el teclado. Al fin y al cabo, los valores almacenados en los archivos de este tipo se encuentran exactamente en el mismo formato que ha usado hasta ahora para entrar información en sus programas: una secuencia de cadenas de texto. La única diferencia es que estos valores no se piden al usuario durante la ejecución, sino que están escritos todos con anterioridad.

Para procesar datos desde un archivo, la clase Scanner permite utilizar una ruta como parámetro para inicializarla. El siguiente código muestra el esquema básico de inicialización de una variable de este tipo de forma que se lean datos desde un archivo.

Recuerde que para usar la clase Scanner necesita importarla desde el package java.util.

```
1 importe java.io.File; 2 importe
java.util.Scanner;
3 ...
4 Scanner lectorArchivo = new Scanner(File f);
```

El parámetro f se corresponde con cualquier variable de tipo File previamente inicializada correctamente. Al hacerlo, debe tenerse en cuenta que la clase Scanner también se comporta como un tipo compuesto. Por tanto, en el caso de trabajar con diferentes archivos, será necesario inicializar diferentes variables.

Por ejemplo, para inicializar una variable de tipo Scanner de forma que permita leer datos desde el archivo ubicado en la ruta "C:\Programas\Unidad 6\Apartado 2\Ejemplos\Document.txt", debería hacerse:

```
1 importe java.io.File; 2 importe
java.util.Scanner;
3 ...
4 File archivo = new File("C:\Programas\Unidad 6\Apartado 2\Ejemplos\Document.txt")
;
5 Scanner lectorArchivo = new Scanner(archivo );
```

PrintStream

Para escribir datos en un archivo, la clase a utilizar es PrintStream. Al igual que con Scanner, para inicializarla correctamente sólo es necesario indicar la ruta del archivo con el que trabajar. En este caso, al tratarse de escritura, la ruta puede indicar un archivo que puede existir o no dentro del sistema. En caso de indicar un archivo que no exista, se creará uno nuevo. En caso de que ya exista, los datos contenidos dentro del fichero se pierden totalmente y éste queda en blanco, con tamaño igual a 0.

Para usar la clase PrintStream necesita importarla desde el package java.io.

```
1 importe java.io.File; 2 importe
java.io.PrintStream;
3 ...
4 PrintStream escritorArchivo = new PrintStream(File f);
```

En este caso, para inicializar una variable de tipo PrintStream de forma que permita escribir datos a un nuevo archivo ubicado en la ruta "C:\Programas\Unidad

6\\Apartado 2\\Ejemplos\\Document.txt", habría que hacer lo siguiente. Como puede ver, el esquema es prácticamente idéntico al de Scanner. Pero en este caso se prepara el archivo para la escritura.

```
1 importe java.io.File; 2 importe
java.io.PrintStream;
3 ...
4 File archivo = new File("C:\\Programas\\Unidad 6\\Apartado 2\\Ejemplos\\Document.txt")
;
5 PrintStream escritorArchivo = new PrintStream(archivo );
```

Pero cuidado, ya que el tratamiento secuencial de datos mediante PrintStream sólo permite trabajar con archivos nuevos. Si la ruta especificada se refiere a un archivo que ya existe, su contenido será eliminado totalmente.

2.1.2 Control de errores

Cuando se realizan operaciones de lectura y escritura sobre archivos pueden ocurrir muchas situaciones anómalas. Desde que, simplemente, el archivo no exista en la ruta especificada, hasta que haya algún error en el sistema de entrada/salida del ordenador o el archivo esté corrompido y la operación se vea interrumpida inesperadamente. El tratamiento de datos con archivos es un proceso mucho más delicado de lo que aparenta inicialmente.

El lenguaje Java considera imprescindible tener en consideración todas estas circunstancias especiales, a las que llama excepciones. Para su control, dispone de un mecanismo algo especial que es obligatorio usar siempre, ya que en caso de no hacerlo se indicará que existe un error de compilación. Por tanto, antes de continuar, es necesario conocer cómo funciona el mecanismo de control de excepciones, ya que de lo contrario no le será posible llevar a cabo ningún programa que trate archivos. Esta sección no tiene ninguna intención de explicarlo en su totalidad, sólo pretende mostrarle lo mínimo imprescindible para poder seguir adelante, y trabajar con archivos.

Para empezar, y ver que no es posible compilar un programa que no controle excepciones al trabajar con archivos, pruebe el programa siguiente en su entorno de trabajo. De acuerdo a su código, éste debería abrir un archivo llamado "Docu-ment.txt" que existe la carpeta de trabajo de la aplicación.

```
1 package unidad6.apartado2.ejemplos; 2 importe java.io.File; 3 importe
java.util.Scanner; 4 public class AbrirArchivo
{ public static void main(String[] args) {
5
6         ObrirFichero programa = new ObrirFichero();
7         programa.inicio();
8
9     } public void inicio() { File f =
10         new File("Document.txt"); Scanner lector = new
11         Scanner(f);
```

```
12     System.out.println("Archivo abierto correctamente.");  
13 }  
14 }
```

Al hacerlo, podrá ver cómo el compilador de Java se queja de un error de este estilo.

```
1 unreported exception java.io.FileNotFoundException; must be caught or declared  
   to be thrown
```

El compilador ha detectado que, dada esta inicialización, puede suceder una excepción en el proceso de entrada/salida, pero el código del programa no lo está controlando (unreported exception...; must be caught, “excepción no declarada ni controlada”, en inglés). Concretamente, avisa que puede darse el caso de intentar abrir un archivo que no existe (FileNotFoundException, “archivo no encontrado”, en inglés). Para evitar este error, todas las instrucciones vinculadas a trabajar con archivos deben estar dentro de lo que se llama una sentencia try/catch.

Una sentencia try/catch se compone de dos bloques de código. Uno para indicar las instrucciones que se deben ir ejecutando mientras no hay excepciones (bloc try), y otro en el momento que se produzca cualquiera (bloc catch).

La sintaxis de este bloque de código tiene cierta similitud con una sentencia if/else, desde la perspectiva de que se crean dos bloques de código. Uno de ellos se ejecuta sólo si se da una condición, y el otro de lo contrario. La sintaxis es la siguiente:

```
1 try { //Código  
2     con las instrucciones para realizar operaciones de entrada / salida. //... 4 } catch (Exception e) {  
3  
4  
5     //Código en caso de que haya un error en cualquier operación anterior //...  
6  
7 }
```

La principal diferencia con un bloc if/else es que no hay ninguna condición lógica a comprobar para decidir qué bloque es necesario ejecutar. Dentro del bloc try se escriben normalmente las instrucciones del programa. Si en cualquier momento durante la ejecución se produce una excepción en alguna operación de entrada/salida, las instrucciones dentro de este bloc dejan de ejecutarse de inmediato, y el flujo de control del programa salta a la primera instrucción del bloc catch. Entonces, se ejecutan todas las instrucciones dentro de este bloc. Si no se produce ninguna excepción en el bloc try, nunca se ejecuta el bloc catch. Por tanto, dentro del bloc catch se ponen las instrucciones para tratar de que ha sucedido un error.

Partiendo de la traducción del nombre de la sentencia del inglés, básicamente dice “intenta (try) hacer esto, y si alguna vez existe alguna excepción durante el proceso, la capturas (catch) y la tratos.”

La forma más gráfica de ver este hecho es directamente con un ejemplo de código.

Escriba el siguiente programa en su entorno de trabajo. Ante todo, verá que ya no hay ningún error de compilación, ya que la instrucción para inicializar Scanner ahora está dentro de una sentencia try/catch. Una vez escrito, ejecútelo sin que allí



La sentencia catch captura la excepción, por tratarla. Fuente: Rdikeman

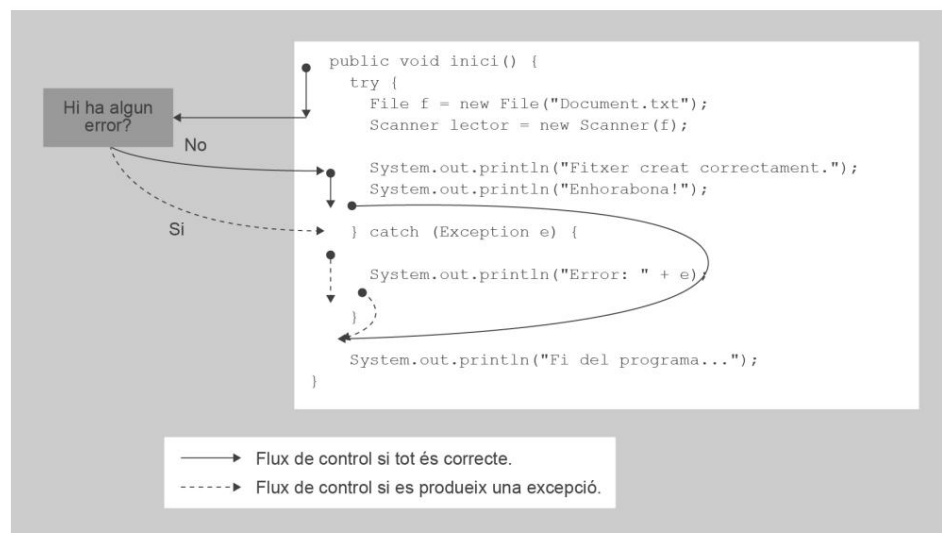
haya un archivo con ese nombre, y en caso contrario. Verá como el resultado del programa será distinto. La figura 2.1 muestra un esquema del flujo de control del programa en cada caso.

```

1 package unidad6.apartado2.ejemplos;
2 importe java.io.File;
3 importe java.util.Scanner;
4 public class AbrirArchivo {
5     public static void main (String[] args) {
6         ObrirFichero programa = new ObrirFichero();
7         programa.inicio();
8     }
9     public void inicio() {
10        try {
11            //Blog "try": operaciones de entrada / salida
12            //Se intenta abrir un archivo para lectura
13            File f = new File("Document.txt");
14            Scanner lector = new Scanner(f);
15            //El archivo existe. Todo correcto
16            System.out.println("Archivo abierto correctamente.");
17            System.out.println("¡Enhorabuena!");
18        } catch (Exception e) {
19            //Blog "catch": Tratamiento de errores
20            //Se ha producido una excepción en algún sitio del blog "try"!
21            System.out.println("Error: " + e);
22        }
23        //Las instrucciones fuera de la sentencia "try/catch" siempre se ejecutan
24        //Igual que las que hay después de una sentencia "if/else"
25        System.out.println("Finalización del programa...");
26    }
27 }

```

Figura 2. 1. Esquema de flujo de control frente a un bloque try-catch.



No deberían llevarse a cabo
operaciones de entrada /
salida en un blog catch

Si se fija en el ejemplo, éste también muestra una particularidad de la sintaxis de esta sentencia. Al inicio del bloque catch hay una variable declarada, con el identificador e, de forma muy parecida a cómo se haría con el parámetro de un método: (Exception e). Esta variable pertenece a la clase Exception, que está en el package java.lang (y por tanto, no hace falta importar).

Las variables de la clase `Exception` contienen información sobre la excepción que ha desencadenado el `try/catch` al que pertenecen. Java se encarga de inicializarlas correctamente siempre que se produzca alguna excepción.

Por tanto, sólo tiene que preocuparse de que ya dispone de esta variable para poder describir qué error se ha producido exactamente. Esta descripción es siempre una cadena de texto, que puede ser impresa de la misma forma que se haría con una variable de tipo `String`. Cuando se produce una excepción, si el código afectado está dentro del método `inicio()`, normalmente es suficiente con mostrar el mensaje de error por pantalla.

Ahora bien, si el código está dentro de otro método auxiliar, y éste en su declaración indica que tiene un parámetro de salida, debe tenerse en cuenta que dentro del código es siempre obligatorio invocar la sentencia `return`, tanto si ha sucedido una excepción como si no. De lo contrario, Java da un error de compilación. Dado este hecho, será necesario que dentro del `try/catch` se invoque `return` con algún valor que se considere no válido, y que sea a posteriori cuando se compruebe si el valor devuelto al invocar este método es válido o no.

A medida que va viendo nuevos aspectos del tratamiento de ficheros a lo largo del apartado, se verá más claro el funcionamiento de la sentencia `try/catch` y el tratamiento de excepciones, especialmente en lo que se refiere a la detección de errores a métodos auxiliares.

2.1.3 Lectura de datos

Desde el punto de vista de instrucciones a ejecutar, la lectura secuencial de datos desde un archivo orientado a carácter es prácticamente igual a leer datos desde el teclado. Los valores se van leyendo de forma ordenada, desde el inicio de la secuencia que representan hasta el final. De hecho, ahora que ya conoce qué es una clase, y que éstas se pueden comportar como tipos compuestos, cómo inicializarlas sobre una variable y cómo invocar métodos correctamente a partir de esta variable, es un buen momento para recapitular sobre el funcionamiento de la clase `Scanner`, con vistas a mostrar cómo llevar a cabo la lectura de datos desde un archivo.

Las operaciones de lectura usando `Scanner` se llevan a cabo mediante la invocación de los métodos que ofrece, resumidos en la tabla 2.1, los cuales evalúan el valor recién leído.

Tabla 2. 1. Métodos de lectura de datos de la clase "Scanner"

Método	Tipo de dato leído
<code>nextByte()</code>	<code>byte</code>
<code>nextShort()</code>	<code>short</code>
<code>nextInt()</code>	<code>int</code>
<code>nextLong()</code>	<code>long</code>
<code>nextFloat()</code>	<code>float</code>
.	

Tabla 2. 1 (continuación)

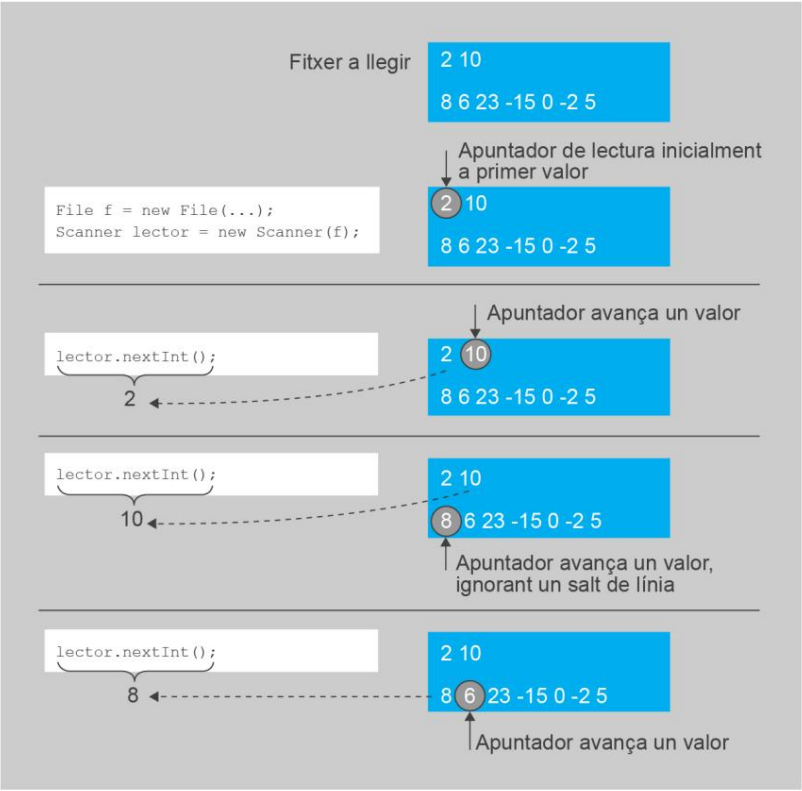
Método	Tipo de dato leído
nextDouble()	double
nextBoolean()	boolean
next()	String (una palabra)
nextLine()	String (una frase)

Una vez leído un valor, éste no puede ser "releído".

En el caso de un archivo, la variable de tipo Scanner gestiona internamente un apuntador que siempre indica sobre qué valor actuarán las operaciones de lectura. Cada vez que se realiza una lectura, el apuntador avanza automáticamente hasta el siguiente valor dentro del archivo, y no hay ninguna manera de hacerlo retroceder. Cuando se inicia una variable de tipo Scanner , este apuntador se encuentra en el primer valor dentro del archivo. Este proceso se repite hasta que se han leído tantos valores como se desea. La figura 2.2 muestra un pequeño esquema de este proceso, recalcando cómo avanza el apuntador a la hora de realizar operaciones de lectura sobre un archivo que contiene valores de tipo entero.

Los distintos valores se distinguen al separarse por espacios o saltos de línea.

F igura 2. 2. Lectura secuencial de valores en un archivo orientado a carácter



Una vez se ha mostrado cómo funciona el apuntador interno al leer valores desde un archivo, vale la pena volver a repasar la diferencia entre el método `next()` y `nextLine()`, ya que ambos evalúan una cadena de texto. El primero sólo lee una palabra individual, considerando "palabra" un conjunto de caracteres que no están separados por espacios o saltos de línea. Este conjunto de caracteres pueden ser tanto palabras, tal y como se encontrarían en un diccionario ("casa", "hola", etc.), como valores numéricos expresados en formato texto ("2", "3,14", "1024", etc.). Sin embargo, `nextLine()` lee una frase completa. Concretamente, lo que lee es todo el texto

que existe entre la posición actual del apuntador y el próximo salto de línea. El apuntador se pone entonces al inicio de la línea siguiente.

Una vez finalizada la lectura de los datos del archivo, ya sean todos o sólo una parte, y ya no hace falta leer más, es imprescindible ejecutar un método especial llamado `close()`. Éste indica al sistema operativo que el archivo ya no está siendo utilizado por el programa. Esto es muy importante, ya que mientras un archivo se considera en uso, su acceso puede verse limitado. Si no invoca `close()`, el sistema operativo puede tardar un tiempo en darse cuenta de que el archivo ya no se encuentra en uso aunque su programa ya haya finalizado. Siempre hay que cerrar el archivo una vez ha finalizado todas las operaciones de entrada / salida.

El siguiente programa muestra un ejemplo de cómo leer diez valores enteros desde un archivo llamado "Enters.txt", ubicado en su carpeta de trabajo. Este archivo de texto deberá crearlo, garantizando que contiene exactamente 10 valores enteros, separados entre sí por espacios en blanco o saltos de línea. Fíjese como todas las instrucciones relativas a la lectura del archivo, las operaciones de entrada / salida, son dentro de una sentencia `try/catch`. También, fíjese que al terminar la lectura, se invoca el método `close()`.

```
1 package unidad6.apartado2.ejemplos;
2 importe java.io.File;
3 importe java.util.Scanner;
4 public class LeerEnteros {
5     public static final int NUM_VALORS = 10;
6     public static void main (String[] args) {
7         LeerEnteros programa = new LeerEnteros();
8         programa.inicio();
9     }
10    public void inicio() {
11        try {
12            //Se intenta abrir el archivo
13            File f = new File("Enters.txt");
14            Scanner lector = new Scanner(f);
15            //Si se ejecuta esta instrucción, se ha abierto el archivo
16            for (int i = 0; i < NUM_VALORS; i++) {
17                int valor = lector.nextInt();
18                System.out.println("El valor leído es: " + valor);
19            }
20            //Hay que cerrar el archivo
21            lector.close();
22        } catch (Exception e) {
23            //Excepción!
24            System.out.println("Error: " + e);
25        }
26    }
27 }
```

Una diferencia importante a la hora de tratar con archivos respecto a leer datos del teclado es que las operaciones de lectura no son producto de una interacción directa con el usuario, que es quien escribe los datos. Sólo se puede trabajar con los datos que hay en el archivo y nada más. Esto tiene dos efectos sobre el proceso de lectura.

Por un lado, recuerde que cuando se lleva a cabo el proceso de lectura de una secuencia de valores, siempre hay que tener cuidado de usar el método adecuado al tipo de valor que se espera que venga a continuación. Saber qué tipo de valor se espera es algo que usted ha decidido vosotros a la hora de hacer el programa, pero nada garantiza que, al escribir el archivo de texto, no se haya equivocado sin querer, como puede ocurrir

El sistema operativo dice que un archivo donde se están haciendo operaciones de entrada / salida está en uso o abierto. Si no, se dice estar cerrado.

Los métodos `next()` o `nextLine()` nunca pueden dar error, ya que los valores en un archivo orientado a carácter siempre se pueden considerar cadenas de texto.

cuando escriba datos mediante el teclado. Ahora bien, en este caso, al operar con archivos, y no por el teclado, no existe la opción de pedir simplemente al usuario que lo vuelva a escribir. Por tanto, el programa debería decir que se ha producido un error ya que el archivo no tiene el formato correcto y finalizar el proceso de lectura.

Por otro lado, también debe controlar que nunca lea más valores de los disponibles para leer. En un caso como éste, a la hora de hacer entrada de datos por el teclado, el programa simplemente se bloqueaba, esperando a que el usuario escribiera nuevos valores. Cuando opera con archivos esto no sucede. Intentar leer un nuevo valor cuando el apuntador ya ha superado el último disponible se considera erróneo. Una vez que se llega al final del archivo, ya no queda más remedio que invocar `close()` y finalizar la lectura.

Si el tipo leído no se corresponde con el método invocado (por ejemplo, se ha invocado el método `nextInt()` cuando el apuntador es sobre un real) o se intenta hacer una lectura cuando el apuntador ya ha superado todos los valores del archivo, se producirá una excepción.

Para verlo, intente ejecutar el ejemplo anterior con archivo donde, o alguno de los valores contenidos no es de tipo entero, o el número de valores almacenados es inferior a diez.

Reto 1. Realice un programa que lea sucesivamente 15 valores de tipo real desde un archivo llamado "Reals.txt". Este archivo lo puede hacer usted mismo. El programa debe mostrar cuál de los valores dentro del archivo es el mayor.

2.1.4 Escritura de datos



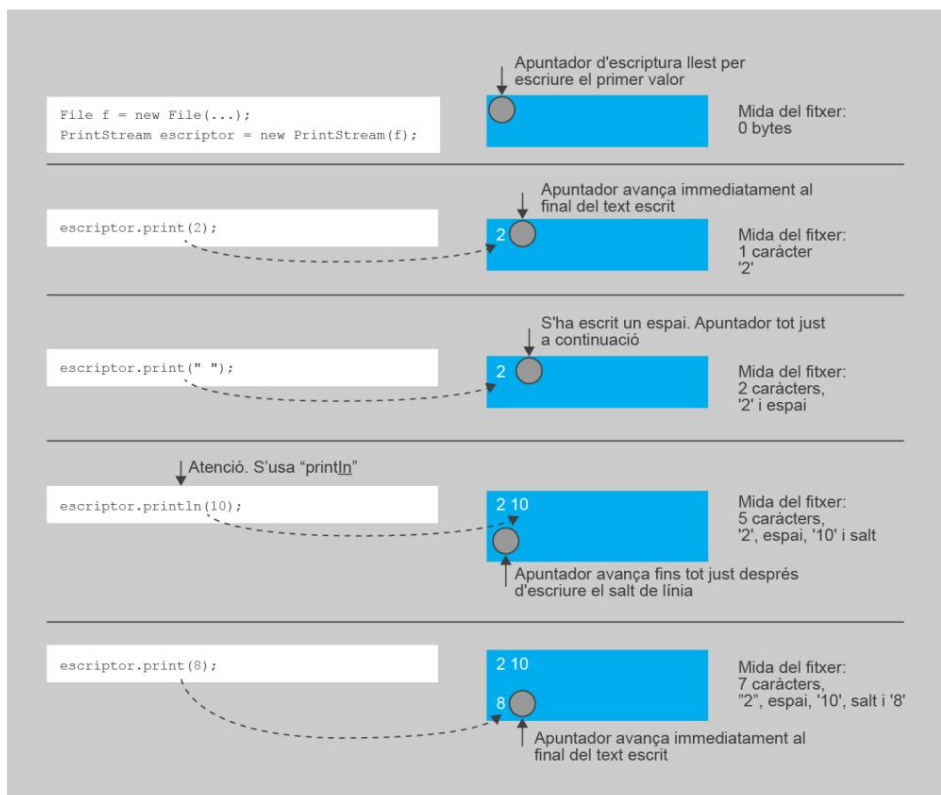
La escritura secuencial de valores también es cómo usar una máquina de escribir.
Fuente: LJL

Recuerde que `PrintStream` sólo opera con archivos nuevos. Si se escribe en un archivo existente, se sobrescribe totalmente.

En este caso, la escritura secuencial de datos en un archivo orientado a carácter es muy parecida a mostrar texto por pantalla. Los datos se van escribiendo una tras otra, con la opción de hacerlo consecutivamente o separados por líneas. Una vez escrito un dato, ya no hay marcha atrás. No es posible escribir información antes o en medio de valores ya escritos.

En este caso, las operaciones de escritura se realizan mediante los métodos proporcionados por la clase `PrintStream`. Estos son básicamente dos, ahora ya viejos conocidos vuestros, que aceptan como único parámetro cualquier tipo primitivo de datos, o una cadena de texto: `print(...)` y `println(...)`. La diferencia es que el primero sólo escribe el valor usado como parámetro en formato cadena de texto, mientras que el segundo también incluye automáticamente un salto de línea, ya que "println" significa "print line" ("imprimir línea", en inglés).

Figura 2. 3. Escritura secuencial de valores en un archivo orientado a carácter



Como en el caso de la lectura, la clase `PrintStream` también gestiona un apuntador que le permite saber a partir de qué posición del texto debe ir escribiendo. Cada vez que se invoca uno de sus métodos de escritura, el apuntador avanza automáticamente, y no es posible recularlo. A efectos prácticos, este apuntador siempre está al final del archivo, por lo que, a medida que se van escribiendo datos, el archivo va incrementando su tamaño. La figura 2.3 muestra un esquema del funcionamiento de las escrituras.

Un aspecto que merece la pena remarcar, y que prevé la figura, es que el proceso de escritura no genera automáticamente el delimitador entre valores. Por tanto, es responsabilidad de su programa escribir los espacios en blanco o salto de línea correspondientes. En caso contrario, los valores quedarán enganchados, y en posterior lectura se interpretarán como un único valor. Por ejemplo, si se escribe el valor entero 2 y después el 4, sin prever ningún delimitador, en el archivo se habrá escrito al final el texto del valor 24.

El siguiente código sirve como ejemplo de un programa que escribe un archivo dentro de su carpeta de trabajo llamado "Enters.txt". Éste contiene 20 valores enteros, de modo que, empezando por el 1, cada uno sea el doble del anterior. Compílelo y pruébelo. Tenga en cuenta que si ya existía un archivo con ese nombre, quedará totalmente sobrescrito. Después, puede intentar leerlo con el programa de ejemplo anterior para leer 10 valores enteros y mostrarlos por pantalla.

```
1 package unidad6.apartado2.ejemplos; 2 importe java.io.File; 3 importe
java.io.PrintStream; 4 public class
EscribirEntersDobles {
5     public static final int NUM_VALORS = 20;
```

```
6 public static void main (String[] args) {  
7     EscribirEntersDobles programa = new EscribirEntersDobles();  
8     programa.inicio();  
9 }  
10 public void inicio() {  
11     try {  
12         //Se intenta abrir el archivo  
13         File f = new File("Enters.txt");  
14         PrintStream escritor = new PrintStream(f);  
15         //Se escribe el primer valor, que no necesita un espacio antes  
16         int valor = 1;  
17         escritor.print(valor);  
18         //Se van generando el resto de valores y escribiendo  
19         for (int i = 1; y < 20; y ++){  
20             //Se escribe antes como delimitador un espacio en blanco  
21             escritor.print(" ");  
22             //Se calcula y escribe el nuevo valor  
23             valor = valor*2;  
24             escritor.print(valor);  
25         }  
26         //Hay que cerrar el archivo  
27         escritor.close();  
28         System.out.println("Archivo escrito satisfactoriamente.");  
29     } catch (Exception e) {  
30         //Excepción!  
31         System.out.println("Error: " + e);  
32     }  
33 }  
34 }
```

El proceso de actualización de los datos se conoce como flush ("tirar de la cadena", en inglés).

Al terminar la escritura de un archivo también es imprescindible invocar el método close() sobre la variable tipo PrintStream.

En el caso de las operaciones de escritura, el cierre del archivo es todavía muy más importante. Esto se debe a que los sistemas operativos a menudo llevan a término de forma diferida la actualización de los datos. O sea, el hecho de ejecutar una instrucción de escritura no significa que de inmediato estos datos ya están escritas en el archivo. Puede pasar un intervalo de tiempo variable entre ambas acciones. Sólo a la hora de ejecutar el método close() se fuerza a que todos los datos pendientes de escribir se actualicen definitivamente en el archivo.

Reto 2. Modifique el ejemplo de modo que los valores enteros, en lugar de estar todos escritos en una sola línea en el archivo de texto, estén divididos en 4 líneas. En cada línea debe haber escritos 5 valores.

2.2 Aspectos importantes del acceso secuencial

Una vez se ha presentado cómo llevar a cabo el acceso secuencial a un archivo orientado a carácter, y ya sabe exactamente cómo se materializa en sentencias de Java, vale la pena exponer algunas situaciones o problemáticas que se pueden encontrar y que pueden complicar el proceso. El objetivo de este apartado es indicar algunos esquemas o estrategias que puede prever para llevar a cabo correctamente la lectura o la escritura de un archivo de este tipo.

2.2.1 Detección de final de secuencia

Aunque el proceso de lectura secuencial de archivos es bastante parecido a la lectura de datos desde el teclado, una de las diferencias sustanciales que hay que tener muy en cuenta es que el número de valores que se pueden leer está fijado por el propio contenido de archivo. Nunca se dará el caso de que el programa se quede esperando a que el usuario escriba texto. Si se intenta leer valores de un archivo cuando ya se ha llegado al final de la secuencia y se han agotado los valores contenidos, se producirá una excepción. Aunque mediante el control de excepciones se puede detectar y tratar este caso, en general, es mejor generar código dentro de sus programas que evite llegar a esta situación. Por eso, dispone básicamente de tres estrategias:

1. Número de valores conocidos a priori

El número de valores dentro del archivo ya es conocido dentro del programa. Por ejemplo, si se espera que el archivo tenga 9 valores, el programa sólo debería hacer 9 lecturas y ninguna. Si se hacen menos, no pasará nada, aparte de que dejaremos valores sin leer, pero si se hacen más, se producirá una excepción. Éste es el caso del ejemplo de lectura que se ha mostrado.

2. Usar una marca de finalización

El último valor no representa realmente un dato a tratar, sino que es un valor especial que identifica el final de la secuencia. Este valor no se considera como parte de los valores a tratar dentro del archivo, sólo es una marca de finalización. Ahora bien, la estrategia sólo vale si hay valores que nunca se darán dentro de la secuencia, ya que de lo contrario es imposible distinguir entre un valor que indica la marca de finalización, o uno que realmente es válido y es necesario tratar. Por ejemplo, si se sabe que todos los valores serán siempre entre -50 y 50, se podría usar un valor -100 como marca de finalización, tal y como muestra el siguiente ejemplo de archivo. Si el archivo pudiera prever cualquier valor, entonces esta estrategia no le sirve.

```
1 2 10 8 6 23 -15 0 -2 5 -100
```

El siguiente programa sería la adaptación del ejemplo de lectura de datos desde un archivo llamado "EntersMarca.txt" partiendo de esta estrategia. Pruébalo usando archivos con diferentes valores escritos. En este caso, la siguiente marca es el valor entero -100.

```
1 package unidad6.apartado2.ejemplos; 2 importe
java.io.File; 3 importe
java.util.Scanner; 4 public class
LeerMarcaFi { public static final int MARCA_FI
5     = -100; public static void main (String[] args) {
6
7     LeerMarcaFi programa = new LeerMarcaFi(); programa.inicio();
8
9
10    } public void inicio() { try { //Se
11        intenta
12        abrir el archivo File f = new
13        File("EntersMarca.txt"); Scanner lector = new Scanner(f);
14
```

```

15         //Esta estrategia se basa en un semáforo
16         boolean leer = true;
17         //Si se ejecuta esta instrucción, se ha abierto el archivo
18         while (leer) {
19             int valor = lector.nextInt();
20             if (valor == MARCA_FI) {
21                 //Marca de fin. Ya ha terminado la lectura
22                 leer = false;
23             } else {
24                 //Aún no ha terminado. Tratar dato
25                 System.out.println("El valor leído es: " + valor);
26             }
27         }
28         //Hay que cerrar el archivo
29         lector.close();
30     } catch (Exception e) {
31         //Excepción!
32         System.out.println("Error: " + e);
33     }
34 }
35 }

```

3. Número de valores indicado en el propio archivo

El primer valor de toda la secuencia puede ser un entero que identifica cuántos valores hay escritos a continuación, por lo que el programa nunca hará más lecturas de las establecidas. Por ejemplo, en el siguiente archivo, el primer valor indica que todo seguido hay 9 enteros. Por tanto, un programa no debería intentar leer más de los indicados, o está garantizado que se va a producir una excepción.

```
1 9 2 10 8 6 23 -15 0 -2 5
```

El siguiente programa sería la adaptación del ejemplo de lectura de datos desde un archivo llamado "EntersMida.txt" partiendo de esta estrategia. Pruébalo usando archivos con diferentes valores escritos.

```

1 package unidad6.apartado2.ejemplos;
2 import java.io.File;
3 import java.util.Scanner;
4 public class LeerMida {
5     public static void main (String[] args) {
6         LeerMida programa = new LeerMida();
7         programa.inicio();
8     }
9     public void inicio() {
10         try {
11             //Se intenta abrir el archivo
12             File f = new File("EntersMida.txt");
13             Scanner lector = new Scanner(f);
14             //Se lee el tamaño
15             int tamaño = lector.nextInt();
16             System.out.println("Hay " + tamaño + " valores.");
17             //Si se ejecuta esta instrucción, se ha abierto el archivo
18             for (int i = 0; i < tamaño; i++) {
19                 int valor = lector.nextInt();
20                 System.out.println("El valor leído es: " + valor);
21             }
22             //Hay que cerrar el archivo
23             lector.close();
24         } catch (Exception e) {
25             //Excepción!
26             System.out.println("Error: " + e);
27         }
28     }
29 }

```

Evidentemente, estos mecanismos no son inmunes a los errores de programación, ya que puede suceder que se despiste y que las cosas no encajen. Quizás se ha olvidado de escribir todos los valores que toca en el archivo, que por algún motivo se haya truncado o corrompido y se haya perdido la marca de finalización, o que el número de valores escrito al principio no sea correcto. Por eso es importante tratar las excepciones y mostrar el error por pantalla, ya que al operar con archivos, nunca se puede estar del todo seguro que los datos que se quieran leer tendrán un formato correcto.

2.2.2 Procesamiento de los datos leídos

Una de las particularidades del tratamiento secuencial de datos desde un archivo es que, una vez inicializada la variable a usar para acceder a ella, los datos sólo se pueden leer una tras otra, pero no es posible retroceder. El apuntador del archivo sólo avanza cada vez que se realiza una operación, pero no puede retroceder. Esto significa que se debería de plantear el procesamiento de los datos de forma que con un único recorrido el programa ya sea posible realizar la tarea en cuestión. Esto normalmente lleva a dos planteamientos posibles.

1. Tratamiento a medida que se lee

En algunos casos, las tareas que debe llevar a cabo el programa se basan en un único recorrido. Se puede establecer un cierto paralelismo entre una situación en la que se parte de un array donde están dispuestos ya los datos, y vosotros para tratarlos basta con hacer un recorrido desde el primer al último elemento. En este caso, en lugar de avanzar posición a posición dentro de un array, se avanza valor a valor dentro de un array archivo. Ejemplos de este caso puede ser encontrar el máximo, el mínimo, calcular una media, mostrar los valores que cumplen ciertas condiciones por pantalla, etc.

Por ejemplo, el siguiente programa, que, dado un fichero llamado "Docu-ment.txt" con diferentes palabras, cuenta la media, sin decimales, de vocales por palabra que existe en total (número de vocales / número de palabras). Para saber cuando termina el archivo, se usa la palabra "fin" como marca (que no debe ser tratada). Como puede verse, incluso en casos donde es necesario acumular resultados y hacer código un poco complejo usando diseño descendente se pueden llevar a cabo las tareas de una sola pasada, si se plantean correctamente.

```
1 package unidad6.apartado2.ejemplos;
2 importe java.io.File;
3 importe java.util.Scanner;
4 public class MediaVocales {
5     //la palabra "fi" sirve como marca de final de archivo
6     public static final String MARCA_FI = "fin";
7     public static void main (String[] args) {
8         MediaVocales programa = new MediaVocals();
9         programa.inicio();
10    }
11    public void inicio() {
12        try {
13            //Se intenta abrir el archivo
14            File f = new File("Document.txt");
15            Scanner lector = new Scanner(f);
16            //Datos a recopilar
```

```
17         int numPalabras = 0;
18         int numVocales = 0;
19         //Marca de fin de lectura
20         boolean leer = true;
21         while (leer) {
22             String palabra = lector.next();
23             if (MARCA_Fl.equals(palabra)) {
24                 //Marca de final
25                 leer = false;
26             } else {
27                 //Tratar dato
28                 numPalabras++;
29                 numVocales = numVocales + contarVocales(palabra);
30             }
31         }
32         System.out.println("Hay" + numPalabras + " palabras.");
33         System.out.println("Hay" double media + numVocales + " vocales.");
34         = numVocals/numPalabras;
35         System.out.println("La media es " + media);
36         //Hay que cerrar el archivo
37         lector.close();
38     } catch (Exception e) {
39         //Excepción!
40         System.out.println("Error: " + e);
41     }
42 }
43 /** Cuenta las vocales en una palabra.
44  *
45  * @param palabra Cadena de texto donde es necesario contar las vocales
46  * @return Número de vocales
47  */
48 public int contarVocales(String palabra) {
49     int res = 0;
50     //Se pasa a minúscula todo para hacerlo más fácil
51     palabra = palabra.toLowerCase();
52     for(int i = 0; i < palabra.length(); i++) {
53         if (esVocal(palabra.charAt(i))) {
54             nada++;
55         }
56     }
57     return nada;
58 }
59 /** Dice si, dado un carácter en minúscula, éste es o no una vocal.
60  *
61  * @param c Carácter a comprobar
62  * @return Si es (true) o no (false) una vocal
63  */
64 public boolean esVocal(char c) {
65     switch(c) {
66         case 'a':
67         case 'e':
68         case 'i':
69         case 'o':
70         case 'u': return true;
71         default: return false;
72     }
73 }
74 }
```

2. Primer lectura y tratamiento posterior

Otra opción disponible es primero ir almacenando los datos leídos dentro un array a medida que se leen, y posteriormente operar sobre ese array.

Esta estrategia tiene la desventaja de que sus programas son algo menos eficientes, ya que primero hacen la lectura de datos y después realmente se dedican a hacer su trabajo. Además, ocupan más memoria, ya que hay que declarar el array on guardar todos los datos del archivo.

Desgraciadamente, hay situaciones en las que es inevitable hacerlo de esta manera. Cualquier situación donde antes de tratar los datos es necesario realizar alguna modificación que depende de todas ellas, habrá que hacerlo de esta forma. Muchos programas de ordenador precisamente por eso usan esta estrategia. Por ejemplo, los editores de texto u hojas de cálculo hacen precisamente esto. Primero cargan el archivo a memoria y posteriormente lo muestran por pantalla de forma que le permiten modificarlo. Hasta que no haga “guardar”, no se actualizan los datos de memoria a disco.

El siguiente ejemplo es otro caso, mucho más sencillo que un procesador de texto, en el que no se puede realizar el trabajo requerido sin cargar antes los datos a un array. Lee los datos de un archivo llamado “Enters.txt”, los ordena, y entonces los escribe en un nuevo archivo llamado “EntersOrdenat.txt”. Para ordenar un conjunto de valores, es necesario disponer de todos ellos a la vez. No hay otra forma. El programa se basa al indicar en su primer valor el número de elementos del archivo para saber cuántos es necesario leer. Fíjese también cómo, mediante diseño descendente, se distribuye el código para tratar cada archivo en métodos diferenciados.

También fíjese que este es un caso en el que se hace tratamiento de excepciones en un método auxiliar, a leerDatos. En un caso como éste, normalmente no se muestra directamente el error por pantalla, sino que se invoca la sentencia return con un valor inválido, y es inicio quien comprueba si este método ha terminado correctamente o incorrectamente. Esto se debe a que todo método declarado con un parámetro de salida siempre debe devolver algún valor, tanto si sucede una excepción como si no.

```
1 package unidad6.apartado2.ejemplos;
2 importe java.io.File;
3 importe java.io.PrintStream;
4 importe java.util.Arrays;
5 importe java.util.Scanner;
6 public class OrdenarArchivo {
7     public static void main (String[] args) {
8         OrdenarArchivo programa = new OrdenarFichero();
9         programa.inicio();
10    }
11    public void inicio() {
12        //Se leen los valores
13        int[] valores = leerDatos("Enters.txt");
14        if (valores != null) {
15            //Se han podido leer los datos correctamente
16            //Se ordenan
17            Arrays.sort(valores);
18            //Se escriben en un nuevo archivo
19            escribirArray("EntersOrdenat.txt", valores);
20        } else {
21            //Ha sucedido un error leyendo los datos
22            System.out.println("Ha habido un error leyendo los datos.");
23        }
24    }
25    /** Dado el nombre de un archivo, lee sus valores y lo carga en un
26        array
27        * de enteros.
28        * @param nombre Nombre del archivo
29        * @return Array con los datos cargados desde el archivo
30        */
31    public int[] leerDatos(String nombre) {
32        try {
33            File f = new File(nombre);
34            Scanner lector = new Scanner(f);
35            int tamaño = lector.nextInt();
36            int[] datos = new int[tamaño];
37            for (int i = 0; i < tamaño; i++) {
```

```
37         datos[i] = lector.nextInt();
38     }
39     return datos;
40 } catch (Exception e) {
41     //No se han podido leer los datos...
42     return null;
43 }
44 }
45 /** Dado un nombre de archivo y un array de enteros, lo escribe en este archivo.
46  *
47  * @param nombre Nombre de la ruta del archivo destino
48  * @param datos Array con los datos a escribir
49  */
50 public void escribirArray(String nombre, int[] datos) {
51     try {
52         //Se intenta crear el archivo
53         File f = new File(nombre);
54         PrintStream escritor = new PrintStream(f);
55         //Primero se escribe el número de valores
56         escritor.print(datos.length);
57         //Se escriben los valores del array, separados por espacios
58         for (int i = 0; i < datos.length; i++) {
59             escritor.print(" " + datos[i]);
60         }
61         System.out.println("Archivo generado satisfactoriamente.");
62         //Hay que cerrar el archivo
63         escritor.close();
64     } catch (Exception e) {
65         //Excepción!
66         System.out.println("Error escribiendo datos: " + e);
67     }
68 }
69 }
```

Otro caso donde es necesario usar esta estrategia para tratar datos es cuando es necesario modificar alguno de los datos que hay en un archivo orientado a carácter. Por ejemplo, suponga que desea cambiar sólo uno de sus valores, dejando el resto igual. Dado que `PrintStream` elimina los datos de un archivo en cuanto se inicializa, en realidad no es posible modificar un valor concreto. Es necesario reescribir su contenido totalmente, previendo los valores que se quieren modificar al hacerlo.

Por ejemplo, el siguiente programa modifica un archivo orientado a carácter que contiene 10 valores de tipo entero y suma 5 a los valores que existen en las posiciones par. Para ello, será necesario leer todos los valores, almacenarlos en un array, hacer las modificaciones oportunas, y luego sobrescribir el archivo original al completo. Por saber cuántos valores existen, se usa la estrategia de indicarlo al inicio del archivo.

```
1 package unidad6.apartado2.ejemplos;
2 importe java.io.File;
3 importe java.io.PrintStream;
4 importe java.util.Scanner;
5 public class ModificarValoresPares {
6     public static final String NOMBRE_FICHERO = "Enters.txt";
7     public static void main (String[] args) {
8         ModificarValoresPares programa = new ModificarValoresPares();
9         programa.inicio();
10    }
11    public void inicio() {
12        //Se leen los valores
13        int[] valores = leerDatos(NOMBRE_FICHERO);
14        if (valores != null) {
15            //Se han podido leer los datos correctamente
16            //Se modifican
17            for (int i = 0; i < valores.length; i = i + 2) {
```

```
18     valores[i] = valores[i] + 5;
19     }
20     //Se vuelven a escribir todos en el archivo original
21     escribirArray(NOMBRE_FICHERO, valores);
22 } else {
23     //Ha sucedido un error leyendo los datos
24     System.out.println("Ha habido un error leyendo los datos.");
25 }
26 }
27 /** Dado el nombre de un archivo, lee sus valores y lo carga en un
    array
28 * de enteros.
29 * @param nombre Nombre del archivo
30 * @return Array con los datos cargados desde el archivo
31 */
32 public int[] leerDatos(String nombre) {
33     try {
34         File f = new File(nombre);
35         Scanner lector = new Scanner(f);
36         int tamaño = lector.nextInt();
37         int[] datos = new int[tamaño];
38         for (int i = 0; i < tamaño; i++) {
39             datos[i] = lector.nextInt();
40         }
41         return datos;
42     } catch (Exception e) {
43         //No se han podido leer los datos...
44         return null;
45     }
46 }
47 /** Dado un nombre de archivo y un array de enteros, lo escribe a ese archivo.
48 *
49 * @param nombre Nombre de la ruta del archivo destino.
50 * @param datos Array con los datos a escribir
51 */
52 public void escribirArray(String nombre, int[] datos) {
53     try {
54         //Se intenta crear el archivo
55         File f = new File(nombre);
56         PrintStream escritor = new PrintStream(f);
57         //Primero se escribe el número de valores
58         escritor.print(datos.length);
59         //Se escriben los valores del array, separados por espacios
60         for (int i = 0; i < datos.length; i++) {
61             escritor.print(" " + datos[i]);
62         }
63         System.out.println("Archivo generado satisfactoriamente.");
64         //Hay que cerrar el archivo
65         escritor.close();
66     } catch (Exception e) {
67         //Excepción!
68         System.out.println("Error escribiendo datos: " + e);
69     }
70 }
71 }
```

Múltiples accesos al archivo

Se pueden encontrar casos en los que un programa debe realizar muchas operaciones sencillas sobre los datos de un fichero, por lo que cada una de ellas, individualmente, se podría hacer a medida que lo lee el archivo. Por ejemplo, suponga un programa que debe encontrar el máximo, el mínimo, y la media aritmética de los valores en un archivo. En un caso como éste, intentar hacer muchas cosas a la vez en un único pase sobre los datos de un archivo puede llegar a complicar el código, o incluso romper su

diseño descendente, ya que tendrá bloques de código que hacen muchas cosas en paralelo en lugar de realizar una tarea muy concreta y fácil de identificar. Se va calculando todo al mismo tiempo a medida que se leen los datos, de modo que en lugar de tres métodos `calculMaxim`, `calculMinim` y `calculMitjana`, puede terminar con uno solo llamado `calculMinimMaximMitjana`!

Una posible opción en este caso sería hacer múltiples accesos al mismo archivo, por lo que se hace un recorrido secuencial nuevo por cada tarea individual. El archivo se abre, se tratan los valores y se cierra varias veces. Ahora bien, esto no es demasiado eficiente dadas las capacidades de los ordenadores modernos, en los que la memoria es abundante, ya que precisamente el acceso a archivos es uno de los puntos más lentos del sistema. En casos como éste, lo mejor es usar la estrategia de primero leer los valores y guardarlos en un array, y después tratarlo para llevar a cabo cada tarea individual.

Sólo sería aconsejable realizar múltiples recorridos en el caso de querer limitar el gasto de memoria de su programa. Por ejemplo, si realmente los archivos son enormes, o el programa debe funcionar en un ordenador con una cantidad modesta de memoria. A escala de estos materiales, nunca se dará el caso. Al fin y al cabo, recuerde que, actualmente, un simple procesador de texto no tiene problemas para cargar archivos con varios megabytes de información.

2.2.3 Archivos con valores de diferentes tipos

Un aspecto que también merece la pena tener en cuenta es que, aunque en los ejemplos mostrados siempre se ha trabajado con archivos que contienen valores del mismo tipo, nada impide mezclarlos. Simplemente tendrá que tener siempre presente cuál es el formato del archivo que lee, y seguirlo fielmente cuando escriba uno nuevo. En casos como éste, es muy recomendable realizarse un esquema de este formato, para tenerlo siempre presente. Entonces, basta con llamar al método adecuado según el tipo de dato esperado.

Por ejemplo, suponga un programa que muestra la nota media de un conjunto de estudiantes. Por cada estudiante, los valores se guardan en líneas diferentes, para facilitar su comprensión, de acuerdo al siguiente formato. En la última línea se encuentra la palabra “fin” como marca de final.

```
1 Nombre(String) Apellido(String) Número de notas(entero) Nota1(real) ... NotaN(real)
```

Por ejemplo:

```
1 Maria Almerich 3 4 3,5 6
2 Juan Sebastián 5 4,5 8,5 5 6,5 7
3 Carme González 4 6,5 8,75 10 9,5 4 fin
```

El código que lo resuelve podría ser el siguiente. Estudie con atención, ya que puede resultar un poco más complicado, a la hora de leer datos de diferentes tipos

mezclados. El punto más interesante es que una variable de tipo Scanner puede ser usada como parámetro de entrada de un método. En este aspecto, al tratarse de un tipo compuesto, no es diferente de un String o un array.

```
1 package unidad6.apartado2.ejemplos;
2 importe java.io.File;
3 importe java.util.Scanner;
4 public class LeerNotes {
5     //la palabra fin sirve como marca de final de archivo
6     public static final String MARCA_FI = "fin";
7     public static void main (String[] args) {
8         LeerNotes programa = new LeerNotes();
9         programa.inicio();
10    }
11    public void inicio() {
12        try {
13            File f = new File("Notes.txt");
14            Scanner lector = new Scanner(f);
15            boolean leer = true;
16            while (leer) {
17                String nombre = lector.next();
18                if (MARCA_FI.equals(nombre)) {
19                    leer = false;
20                } else {
21                    String apellido = lector.next();
22                    System.out.print("Estudiante: " + apellido);    + nombre + "
23                    //Note cómo un scanner se puede pasar como parámetro
24                    double media = leerNotes(lector);
25                    System.out.println("– Media: " + media);
26                }
27            }
28            //Hay que cerrar el archivo
29            lector.close();
30        } catch (Exception e) {
31            //Excepción!
32            System.out.println("Error leyendo estudiantes: " + e);
33        }
34    }
35    /** Dado un Scanner en un archivo cuando el apuntador se encuentra sobre el inicio
36     * de las notas, se extraen y se calcula la media.
37     *
38     * @param lector Scanner a procesar
39     * @return Media de notas
40     */
41    public double leerNotes(Scanner lector) {
42        double nada = 0;
43        try {
44            //Se hacen lecturas. ¡Hay que controlar excepciones también!
45            int numNotes = lector.nextInt();
46            for (int i = 0; i < numNotes; i++) {
47                //Se acumula el valor de las notas
48                nada = nada + lector.nextDouble();
49            }
50            //Se calcula nota media
51            nada = nada/numNotes;
52        } catch (Exception e) {
53            //Excepción!
54            System.out.println("Error leyendo notas: " + e);
55        }
56        return nada;
57    }
58 }
```

Reto 3. Modifique el ejemplo de forma que ahora haga lo siguiente. Por un lado, en lugar de mostrar los datos por pantalla, los debe escribir en un archivo llamado "NotaMedia.txt". Por otra parte, se cambia el formato del archivo "Notes.txt". Por controlar el número de notas de cada estudiante, en lugar de indicarlo directamente al

tercer valor de cada línea, ahora se pone al final de cada línea el valor -1. Por ejemplo, ahora debe poder leer el archivo siguiente:

```
1 Maria Almerich 4 3,5 6 -1
2 Juan Sebastián 4,5 8,5 5 6,5 7 -1
3 Carme González 6,5 8,75 10 9,5 -1
4 fin
```

2.3 Acceso secuencial a archivos orientados a byte

En el caso de los archivos orientados a byte, los datos también se almacenan como una secuencia de valores, y por tanto, el esquema general que aplica en ficheros orientados a carácter también puede aplicarse a este otro tipo. Todas las estrategias para saber cuántas lecturas es posible llevar a cabo también aplican. La diferencia fundamental es que no se utiliza su representación en forma de cadena de texto, sino que se usa su formato binario, directamente tal y como se usa dentro de la memoria del ordenador. Según el tipo de datos del valor almacenado, éste se almacenará dentro de un número determinado de bytes. Además, los archivos orientados a byte no usan ningún delimitador para separar los valores.

La tabla 2.2 muestra una recopilación del tamaño de las representaciones en binario de los diferentes tipos primitivos de Java.

Tabla 2. 2. Espacio necesario para almacenar los distintos tipos primitivos

Tipo	Palabra clave Java	Tamaño (bytes)
carácter	char	normalmente 2
byte	byte	1
entero corto	short	2
entero simple	int	4
entero largo	long	8
real de simple precisión	float	4
real de doble precisión	double	8

La representación binaria de los enteros en Java es el complemento a dos.

Dado este hecho, suponga que desea almacenar dentro de un archivo de éstas características los valores enteros 2, -14 y 25. Esto significa que dentro del archivo se guardan de acuerdo a su representación binaria en 4 bytes (32 bits), que sería la siguiente para Java:

- 2: 00000000000000000000000000000010
- -14: 11111111111111111111111111110010
- 25: 000000000000000000000000000011001

Para inspeccionar un archivo orientado a byte es necesario un editor hexadecimal, como el HexEdit (Windows) o GHex (Unix).

La figura 2.4 muestra un esquema de cuál sería la estructura de un archivo orientado a byte que contiene estos tres valores. Primero se muestra su equivalente en archivo

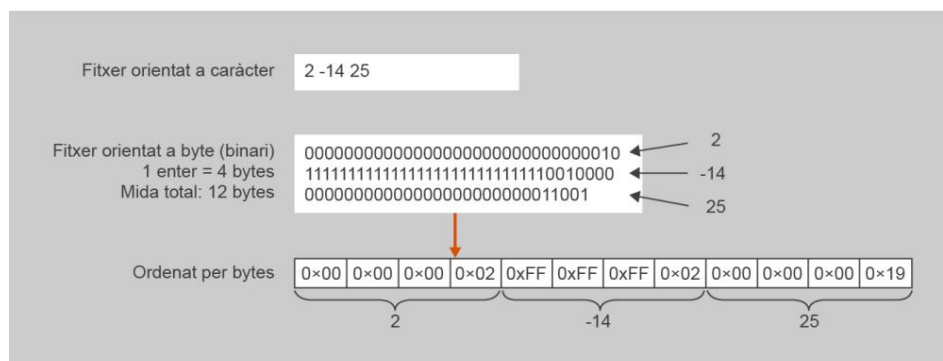
orientado a carácter y cuál sería el contenido directamente en formato binario (los tres conjuntos de 0 y 1 recién presentados, uno tras otro). Aunque, por motivos de espacio en la figura, cada valor está en una fila diferente, dentro de un archivo estarían escritos como una secuencia completa. No hay saltos de línea. Finalmente, en la figura se muestra en un formato en el que se diferencia el valor de cada byte, por lo que es más fácil seguir su contenido, contando que cada entero se representa con 4 bytes.

La notación hexadecimal

Trabajar con notación binaria es algo farragoso, ya que los valores se codifican en secuencias de 0 y 1 que pueden ser largas (32-64 bits), por lo que ocupan bastante espacio y resultan difíciles de leer. Normalmente, para facilitar la comprensión, los valores se representan en sistema hexadecimal, por lo que quedan mejor resumidos y son fáciles de dividir en bytes. En el sistema hexadecimal, los dígitos van del 0 al 15, contando que los valores del 10 al 15 se representan con las letras A a la F. Cada dígito representa 4 bits, de modo que 0 = 0000, 1 = 0001 ... E = 1110, F = 1111.

Cuando un valor quiere representarse en hexadecimal, se usa el prefijo "0x". Éste no es parte del valor, sólo es un símbolo para indicar esto.

Figura 2. 4. Estructura de un archivo orientado a byte



Tenga en cuenta que cuando se dice que los datos se representan en binario, se quiere decir exactamente esto. No se trata de que en el archivo se escribe texto con los caracteres '1' y '0'. Si intenta abrir un archivo orientado a byte con un editor de texto, no visualizará nada. A modo de ejemplo, la mayoría de archivos de los procesadores de texto o de hojas de cálculo están orientados a byte. Por tanto, si los abra con un editor de texto simple, lo que se visualizará será totalmente incomprensible (además, sufra el riesgo de dañar los datos contenidos).

Si se fija en la figura, cuando los valores se resumen separando sus bytes en hexadecimal, un archivo orientado a byte tiene una estructura similar a la de un array, donde cada posición sería representada por cada uno de sus bytes, y cada valor concreto en realidad ocupa un cierto número de posiciones. Esta característica debe tenerla presente, ya que será muy relevante más adelante.

Todo esto significa que no es posible generar archivos orientados a byte sólo con un editor de texto, ni tampoco son fáciles de inspeccionar a simple vista. La principal ventaja de un archivo de este tipo es que, a costa de una cierta complicación a la hora de controlar su contenido, todos los valores de un mismo tipo siempre ocuparán exactamente el mismo espacio dentro del archivo. En muchos casos, esto significa que el archivo ocupa menos espacio. Por ejemplo, si se desea almacenar los valores de

tipo entero 15 y 10371, en un archivo orientado a carácter el primero ocupa 2 caracteres y el segundo cinco. Teniendo en cuenta que se deben separar de un espacio, esto equivale a 8 o 16 bytes, ya que, dependiendo de la configuración de su entorno de trabajo, cada carácter en Java se guarda en 1 o 2 bytes. En el caso de un archivo orientado byte, escribir ambos siempre ocupará 8 bytes.

2.3.1 Inicialización

En este caso, para los archivos orientados a byte se puede usar la misma clase tanto si se quieren leer datos como si se desea escribir.

Para tratar de forma sencilla archivos orientados a byte, Java ofrece la clase `RandomAccessFile`, perteneciente al package `java.io`. Ésta se usa tanto para leer como para escribir datos.

De nuevo, ésta actúa como un tipo compuesto. Por tanto, al igual que ocurre con las clases `File`, `Scanner` o `PrintStream`, es necesario inicializarla con una variable diferente para cada archivo sobre el que se quiere trabajar (en el caso de trabajar con más de un archivo a la vez). La forma de hacerlo es con la siguiente instrucción:

```
1 importe java.io.File; 2 importe
java.io.RandomAccessFile;
3 ...
4 RandomAccessFile raf = new RandomAccessFile(File ruta, String mode);
```

En este caso, para inicializar una variable de este tipo correctamente se necesitan dos parámetros. Por un lado, y al igual que ocurría con `Scanner`, es necesario especificar la ruta del archivo con el que se quiere trabajar. Adicionalmente, es necesario indicar mediante una cadena de texto el modo de trabajo a la hora de procesar los datos del archivo. Hay varios modos de trabajo, pero sólo verá los dos más sencillos, usados en la inmensa mayoría de casos. La utilidad de los demás modos es más marginal.

Estos dos modos son:

- **r:** modo lectura. Indica que sólo se desea leer datos desde el archivo representado por la ruta especificada. Esto implica que la ruta debe referirse a un archivo existente en el sistema de archivos. De lo contrario, se producirá un error. También producirá un error cualquier intento de invocar métodos asociados a la escritura de datos sobre la variable `RandomAccessFile` recién inicializada.
- **rw:** modo escritura-lectura. Este modo permite tanto leer datos desde un archivo, al igual que el anterior, como escribirlos. Si la ruta usada se refiere a un archivo que no existe, se creará uno nuevo, que esté vacío.

Por tanto, si sólo se quieren leer datos de un archivo ubicado en la ruta "C:\Programes\Unidad 6\Apartado 2\Ejemplos\Document.txt", habría que hacer:

```
1 importe java.io.File; 2 importe  
java.io.RandomAccessFile;  
3 ...  
4 File f = new File("C:\\Programas\\Unidad 6\\Apartado 2\\Ejemplos\\Document.txt"); 5 RandomAccessFile raf = new  
RandomAccessFile(f, "r");
```

Si, por el contrario, se quieren escribir datos a un archivo en esta misma ubicación,

habrá que hacer:

```
1 importe java.io.File; 2 importe  
java.io.RandomAccessFile;  
3 ...  
4 File f = new File("C:\\Programas\\Unidad 6\\Apartado 2\\Ejemplos\\Document.txt"); 5 RandomAccessFile raf = new  
RandomAccessFile(f, "rw");
```

Al contrario que con la clase `PrintStream`, si la ruta usada al inicializar una variable de esta clase en modo "rw" se corresponde a un archivo que ya existe, no se borran sus datos, ni quedan afectados en ninguna medida.

Puede que le parezca un poco extraño o redundando el modo "r", existiendo el modo "rw", que permite hacer exactamente lo mismo y más cosas. En realidad, usar el modo "r" siempre que sólo se quieren leer datos es bastante útil, ya que garantiza que si, por un error al programar, su programa intenta escribir en el archivo con el que trabaja, la operación de escritura se abortará inmediatamente. Esta posibilidad es especialmente interesante cuando se está trabajando con datos de ficheros, ya que no hace ninguna gracia sobreescribir (y, por tanto, perder) la información que había en un archivo debido a un error involuntario en el código.

Por tanto, a partir de ahora, si sólo se quieren leer datos de un archivo, siempre se usará el modo "r" exclusivamente.

2.3.2 Escritura de datos

En este caso, se empezará por explicar la escritura de datos, en lugar de la lectura. El motivo principal es que, dado que `RandomAccessFile` trabaja con archivos orientados a byte, no hay ninguna manera sencilla de generar archivos de datos por su cuenta, al contrario que para el caso de los archivos orientados a carácter, donde basta con usar un editor de texto. La única forma de generar archivos orientados a byte que puedan ser leídos correctamente es mediante código de un programa.

Cada vez que se realiza una operación de lectura o escritura, el apuntador se desplaza automáticamente el mismo número de bytes con los que se ha operado.

El número de bytes desplazados dependerá del tamaño asociado al tipo primitivo de Java escrito. Hay que tener siempre en cuenta que, como `RandomAccessFile` trabaja con archivos orientados a byte, esta clase transforma el valor del tipo primitivo a una secuencia de bytes, según el método invocado. Para saber exactamente en cuánto ha variado el apuntador del archivo, es necesario saber cuántos bytes ocupa cada tipo primitivo de Java.

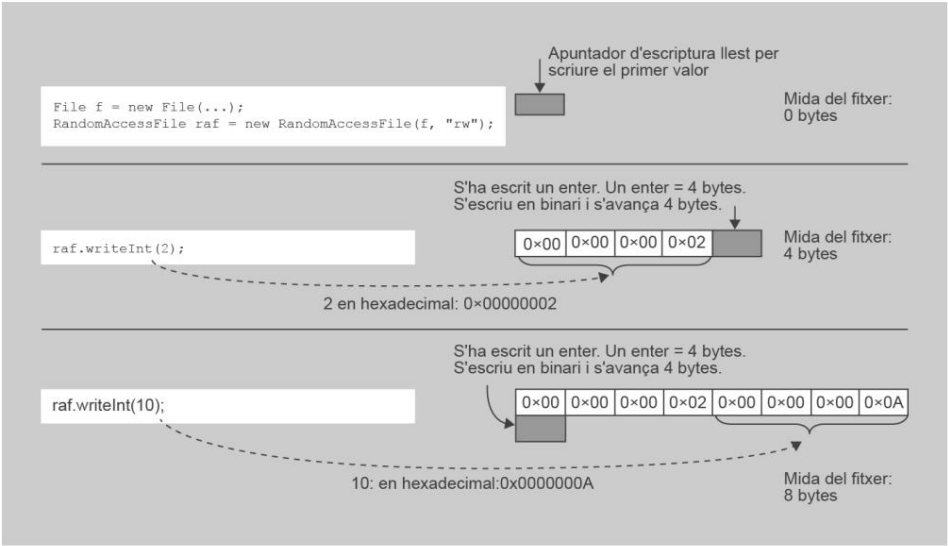
En la tabla 2.3 se muestran los métodos disponibles para escribir datos.

Tabla 2. 3. Métodos de escritura de datos de la clase "RandomAccessFile"

Método	Bytes escritos
writeBoolean(boolean b)	1
writeByte(byte v)	1
writeChar(char c)	2
writeDouble(double d)	8
writeFloat(float f)	4
writeInt(int i)	4
writeLong(long l)	8
writeShort(short s)	2

Además de la diferencia en el formato de representación de los datos del archivo y que no es necesario encargarse de escribir ningún delimitador entre los valores, el proceso general de escritura de datos es similar a la de los archivos orientados a carácter: un valor tras otro. La figura 2.5 muestra un esquema del funcionamiento del proceso de escritura.

F igura 2. 5. Escritura secuencial de valores en un archivo orientado a byte



Como ejemplo, estudie y pruebe el siguiente programa. Ésta es la versión de el ejemplo anterior que escribe una secuencia de 20 valores enteros, por lo que, empezando por el 1, cada valor es el doble del anterior. Fíjese como no es necesario escribir ningún delimitador. Los valores se van escribiendo consecutivamente, uno tras el otro.

```
1 package unidad6.apartado2.ejemplos;
2 importe java.io.File;
3 importe java.io.RandomAccessFile;
4 public class EscribirEntersDoblesBinario {
5     public static void main (String[] args) {
6         EscribirEntersDoblesBinario programa = new EscribirEntersDoblesBinario();
7         programa.inicio();
8     }
9     public void inicio() {
```

```
10     try {
11         File f = new File("Enters.bin");
12         RandomAccessFile raf = new RandomAccessFile(f,"rw");
13         //Ahora no hay delimitadores. Se escriben los valores consecutivos.
14         //Se van generando los valores y escribiendo
15         int valor = 1;
16         for (int i = 0; i < 20; i++) {
17             raf.writeInt(valor);
18             valor = valor*2;
19         }
20         System.out.println("Archivo escrito satisfactoriamente.");
21         //El tamaño de un entero son 4 bytes.
22         //El tamaño del archivo debería ser 20*4 = 80 bytes
23         //No olvidarse de cerrar el archivo
24         raf.close();
25     } catch (Exception e) {
26         //Excepción!
27         System.out.println("Error escribiendo datos: " + e);
28     }
29 }
30 }
```

Sobreescritura de archivos

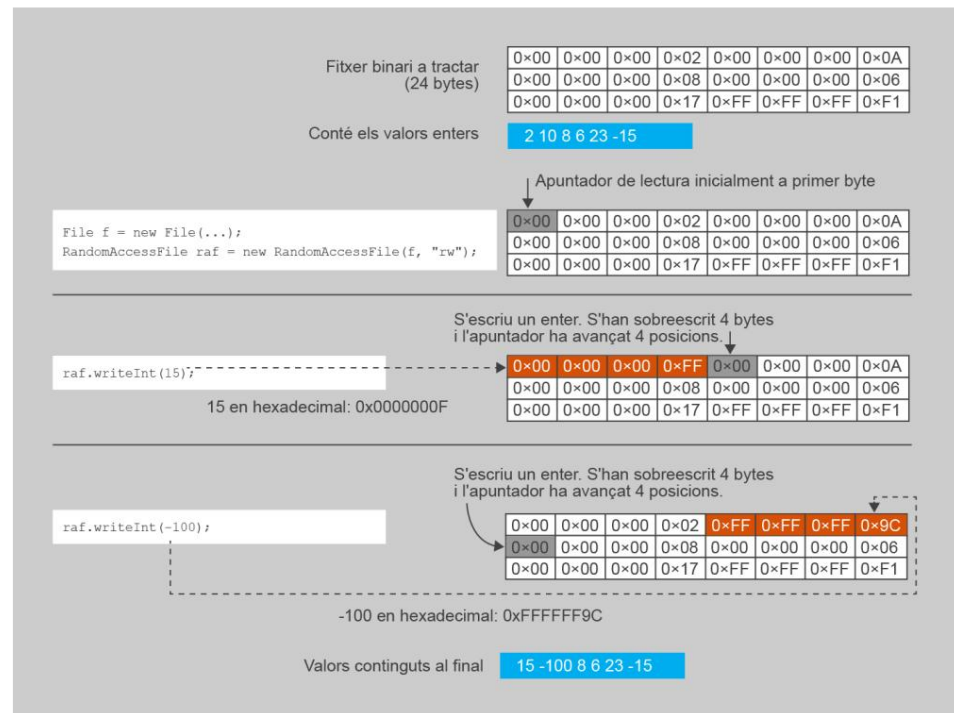
La clase `RandomAccessFile` tiene un comportamiento algo especial cuando escribe datos sobre un archivo que ya existe, en lugar de hacerlo sobre uno completamente nuevo. Al contrario que `PrintStream`, este archivo existente no se borra y se comienza a escribir desde cero, sino que los nuevos datos se van escribiendo directamente sobre las antiguas, reemplazándolas byte a byte. Éste es un hecho con el cual es necesario tener mucho cuidado, ya que si el número de bytes escritos es inferior a la tamaño del archivo existente, quedará “basura” al final (los datos del archivo existente que no se han llegado a sobreescribir).

La figura 2.6 le muestra un esquema de este hecho, en un caso en que existe un fichero con 6 valores enteros escritos y queremos reemplazar su contenido a sólo dos valores, 15 y -100.

Según este comportamiento, es necesario tener mucho cuidado con el hecho de que los datos que no son sobreescritos se mantienen en el archivo, sobre todo cuando se desea reemplazar todo el contenido de un archivo por datos nuevos. Normalmente, lo que se suele hacer en estos casos, es eliminar todos los bytes sobrantes una vez terminada la escritura de datos. Para ello, se dispone de los métodos:

- `setLength(long tamaño)`. Modifica el tamaño del archivo, por lo que si el valor especificado como parámetro es menor que el tamaño actual, se eliminan todos los datos por encima del tamaño especificado. Si es mayor, se llena la diferencia con bytes todo a 0.
- `long getFilePointer()`. Evalúa la posición donde está en estos momentos el apuntador, medido en el número de bytes desde el inicio del archivo. Por al caso de que está tratando, si se invoca cuando ha terminado todas las operaciones de escritura, le dirá el tamaño de los datos escritos, y por tanto, el nuevo tamaño que debe tener el archivo.

Figura 2. 6. Sobreescritura de archivos al usar RandomAccessFile



El programa siguiente usa este mecanismo para reemplazar completamente el archivo resultante del ejemplo anterior (20 valores enteros, cada uno el doble de el anterior), llamado "enters.bin", por cinco valores enteros -1. El resultado final es un archivo de sólo 20 bytes, en lugar de 80. Pruebe también qué sucede si antes de ejecutarlo sobre este archivo de 80 bytes se elimina la sentencia `raf.setLength(apuntador)`.

```
1 importe java.io.File;
2 importe java.io.RandomAccessFile;
3 public class SobreescibirEntersBinario {
4     public static void main (String[] args) {
5         SobreescibirEntersBinario programa = new SobreescibirEntersBinario();
6         programa.inicio();
7     }
8     public void inicio() {
9         try {
10             File f = new File("Enters.bin");
11             RandomAccessFile raf = new RandomAccessFile(f, "rw");
12             //El apuntador está en el primer byte
13             long apuntador = raf.getFilePointer();
14             System.out.println("Inicio: Apuntador a posición" + apuntador);
15             //Sobreescribimos los primeros cinco valores
16             for (int i = 0; i < 5; i++) {
17                 raf.writeInt(-1);
18             }
19             //Si el archivo ya tenía más de cinco enteros, al final hay basura
20             apuntador = raf.getFilePointer();
21             System.out.println("Fin: Apuntador a posición" + apuntador);
22             //Se fija el tamaño del archivo a los valores escritos
23             raf.setLength(apuntador);
24             raf.close();
25             System.out.println("Archivo modificado correctamente.");
26         } catch (Exception e) {
27             //Excepción!
28             System.out.println("Error escribiendo datos: " + e);
29         }
30     }
31 }
```

2.3.3 Lectura de datos

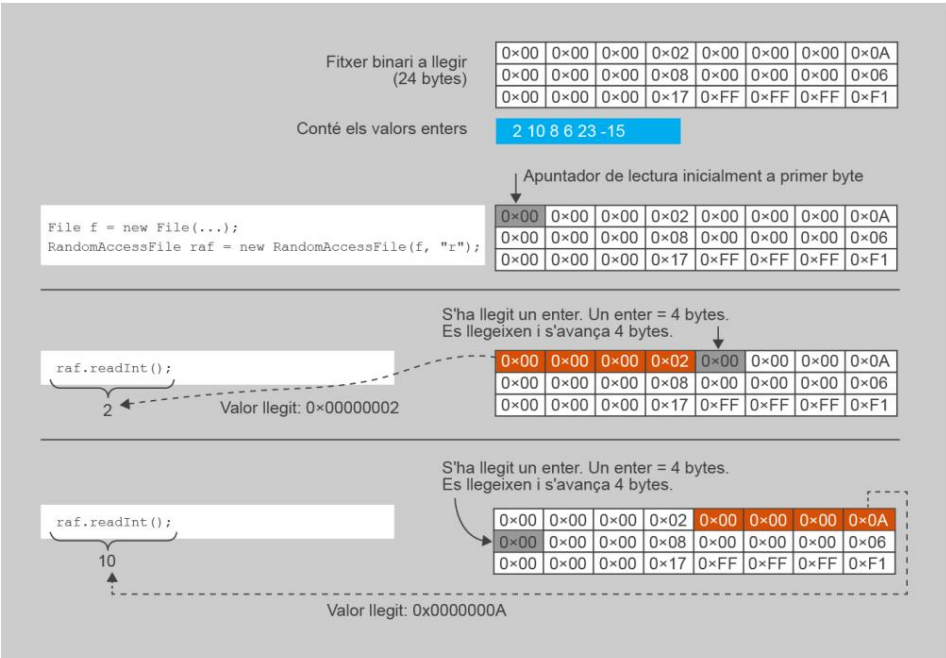
Una vez conocido el proceso de lectura de datos de forma secuencial y el formato como se escriben los datos en un archivo orientado a byte, no hay muchas sorpresas en el proceso de lectura. De nuevo, se parte de un apuntador en la posición inicial que va avanzando a medida que se invocan métodos para realizar operaciones de lectura. Exactamente al igual que ocurría con la clase Scanner, hay un método específico para cada tipo de datos esperado dentro de la secuencia de valores, enumerados en la [tabla 2.4](#).

Tabla 2. 4. Métodos de lectura de datos de la clase "RandomAccessFile"

Método	Tipo de dato leído
readByte()	byte
readShort()	short
readInt()	int
readLong()	long
readFloat()	float
readDouble()	double
readBoolean()	boolean
readChar()	char

La figura 2.7 muestra un ejemplo del funcionamiento del proceso de lectura en un archivo orientado a byte.

F igura 2. 7. Lectura secuencial de valores en un archivo orientado a byte



En el caso de los archivos binarios también es necesario controlar no leer más valores de los que realmente hay escritos en el archivo, o de lo contrario se producirá una excepción. Ahora bien,

en este tipo de archivo tiene una ventaja, y es que no siempre es necesaria una marca de finalización o indicar el número de elementos previamente. Dado que cada valor del mismo tipo siempre ocupa el mismo espacio (los int 4 bytes, los double 8 bytes, etc.), si conoce el tamaño del archivo en bytes, se puede calcular cuántos valores de un tipo contiene haciendo una simple división: tamaño archivo/tamaño tipo.

El siguiente código de ejemplo lee un archivo orientado a byte llamado "Enters.bin", el cual contiene cierto número de valores enteros, y los va mostrando por pantalla. Puede utilizar el resultado de los ejemplos de escritura de datos. Fíjese especialmente en la forma en que se calcula el número de valores a leer, dado el tamaño del archivo y conocido el tamaño de la representación en binario de un entero en Java (4 bytes).

```

1 package unidad6.apartado2.ejemplos;
2 importe java.io.File;
3 importe java.io.RandomAccessFile;
4 public class LeerEntersBinario {
5     public static void main (String[] args) {
6         LeerEntersBinario programa = new LeerEntersBinario();
7         programa.inicio();
8     }
9     public void inicio() {
10        try {
11            File f = new File("Enters.bin");
12            RandomAccessFile raf = new RandomAccessFile(f, "r");
13            //Cálculo del número de enteros
14            long numEnters = f.length() / 4;
15            System.out.println("Hay " + numEnters + " for (int i = 0; i < numEnters; i++) {
16                int valor = raf.readInt();
17                System.out.println("Se ha leído el valor " + valor);
18            }
19            raf.close();
20        } catch (Exception e) {
21            //Excepción!
22            System.out.println("Error en la lectura: " + e);
23        }
24    }
25 }
26 }

```

Reto 4. Haga un programa que lea un archivo orientado a byte que contenga una secuencia de valores reales cualquiera. Hay que mostrarlos por pantalla ordenados de más grande a menor. Tenga en cuenta que, para poder probarlo, antes deberá generar este archivo usted mismo de alguna manera.

Lectura incorrecta de datos

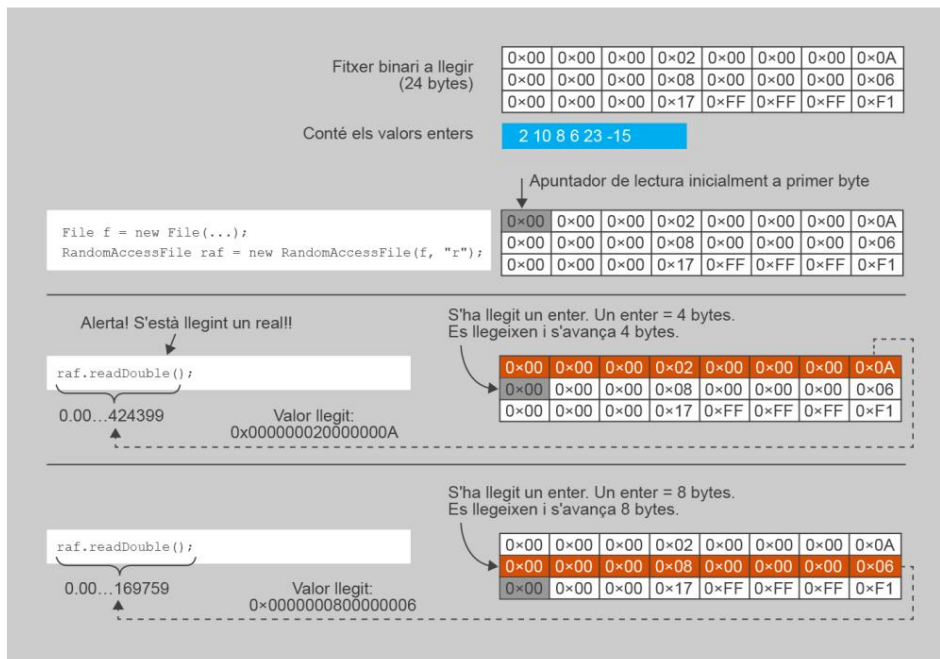
Lectura incorrecta de datos

Es parecido a lo que ocurre si, con un procesador de texto, por ejemplo, intente abrir un archivo que no le corresponde, hecho con un otro programa.

A la hora de leer datos en un archivo orientado a byte, es necesario procurar siempre usar el método acertado en cada momento, de acuerdo al tipo que se espera leer. Ahora bien, la clase `RandomAccessFile` actúa de una forma bastante diferente a `Scanner`, ya que cuando realiza una lectura incorrecta no se produce ninguna excepción. Esta clase, cuando ejecuta cualquiera de sus métodos de lectura, siempre lee el número de bytes asociados al tipo de dato que se quiere leer, interpretándolos al valor que corresponde dentro de este tipo. Esto lo hace independientemente de que, en realidad, esto sea correcto o no.

La figura 2.8 muestra un ejemplo de este comportamiento, leyendo un valor de tipos real sobre un archivo donde hay almacenados valores enteros.

Figura 2. 8. Lectura incorrecta de datos en un archivo orientado a byte



Para verlo más claro, a continuación se muestra una versión del ejemplo anterior que lee y presenta valores reales. En este caso, ejecutarlo sobre un archivo donde sólo hay ha escrito valores enteros sería incorrecto, ya que no se está usando el método adecuado para leer los valores contenidos. Ahora bien, el método `readDouble()` simplemente lee 8 bytes consecutivos (o sea, en este caso, la representación en el fichero de dos enteros seguidos, 4 + 4 bytes), e intenta interpretar estos 8 bytes como si fueran un valor de tipo real. Evidentemente, el valor resultante de realizar este proceso será totalmente incorrecto, ya que dentro del archivo no hay ningún valor de tipo real. Ejecute para ver el resultado.

```
1 package unidad6.apartado2.ejemplos;
2 importe java.io.File;
3 importe java.io.RandomAccessFile;
4 public class LeerDoubleBinario {
5     public static void main (String[] args) {
6         LeerDoubleBinario programa = new LeerDoubleBinario();
7         programa.inicio();
8     }
9     public void inicio() {
10        try {
11            File f = new File("Enters.bin");
12            RandomAccessFile raf = new RandomAccessFile(f, "rw");
13            //Cálculo del número de enteros
14            long numEnters = f.length() / 8;
15            System.out.println("Creo que hay " + numEnters + " reales.");
16
17            double valor = raf.readDouble();
18            System.out.println("Se ha leído el valor real" + valor);
19        }
20        raf.close();
21    } catch (Exception e) {
22        //Excepción!
23        System.out.println("Error en la lectura: " + e);
24    }
25 }
26 }
```

Por tanto, es necesario ser muy cuidadoso a la hora de hacer lecturas sobre archivos orientados a byte, y tener bien claro su formato y qué tipo de datos hay que leer en cada momento, ya que el programa no nos avisará con ninguna excepción. Simplemente, éste comportará de forma extraña al ejecutarlo.

2.4 Acceso relativo a los datos

El acceso secuencial a los datos de un archivo permite procesar su contenido de forma relativamente sencilla, al basarse en un mecanismo prácticamente idéntico a la lectura de datos desde el teclado, o su escritura en la pantalla. Los bloques de instrucciones para llevar a cabo operaciones de lectura y escritura son básicamente los mismos, sólo varía el origen y el destino de los datos en las operaciones de entrada/salida, así como los aspectos vinculados a si el archivo es orientado a carácter o a byte.

Ahora bien, en realidad, un archivo no es exactamente como el teclado o la pantalla, ya que los datos ni dejan de estar disponibles una vez leídos, ni tiene sentido que resulte imposible sobreescribirlos parcialmente. El acceso secuencial es una manera cómoda de tratar archivos, pero a cambio de perder un cierto grado de versatilidad.

Si considera que un archivo orientado a byte, y sólo este tipo, es como un array de cierta envergadura donde cada byte representa una posición, debe ser posible acceder a los datos que contiene de la misma forma que se hace con los arrays. Por un lado, de forma secuencial, haciendo recorridos sobre cada una de sus posiciones, como ha visto hasta ahora. Pero también debe ser posible acceder directamente a una posición concreta sólo indicando una ubicación para su apuntador de lectura o escritura. O sea, dado un archivo orientado a byte, se puede decir que dispone de acceso relativo.

El acceso relativo a un archivo a menudo también se llama acceso aleatorio o acceso directo. Dentro de ese contexto, son términos sinónimos.

Se llama acceso relativo a la capacidad de acceder a cualquier elemento dentro de una secuencia de datos sin tener que tratar previamente los elementos anteriores.

Una buena forma de entender esta definición es contrastando el acceso relativo al secuencial, al ser precisamente términos contrapuestos. Dado un archivo con una secuencia de 10 elementos, para poder tratar sólo el décimo, si se accede secuencialmente, primero hay que leer (e ir descartando sucesivamente) los nueve elementos anteriores. El acceso relativo, en cambio, proporciona operaciones que hacen posible leer ese décimo elemento directamente, sin tener que procesar antes todos los anteriores usando métodos de lectura.

La clase `RandomAccessFile` permite el acceso relativo a los archivos orientados a byte.

Al fin y al cabo, su nombre, en inglés, significa "fichero de acceso aleatorio".

2.4.1 Posicionamiento

El primer paso para poder acceder relativo a un archivo es especificar el índice de la posición que se desea acceder, ya sea tanto para leer el dato que hay almacenado como para escribir un nuevo valor. Una vez conocido este índice, mediante la sintaxis adecuada, es posible esa posición. En el caso de los arrays, esta sintaxis es del tipo:

```
1 array[índice]
```

Dado que `RandomAccessFile` es una clase, la sintaxis para acceder a las posiciones del archivo es mediante la invocación de métodos. Cada byte individual dentro del archivo es como una posición de un array, y el primer byte del archivo se consideraría su posición 0.

Para llevar a cabo la tarea de indicar el índice para realizar una operación de lectura o escritura, las variables de tipo `RandomAccessFile` tienen una diferencia importante en el acceso a los datos respecto a un array. Si lo recuerda, sus métodos de lectura y escritura no disponen de ningún parámetro donde se especifique un índice. Esto se debe a que las clases que tratan los datos de un archivo disponen de un apuntador interno que indica en todo momento cuál es el dato siguiente a procesar. Cada vez que se realiza una operación, este apuntador se va desplazando automáticamente hasta llegar al final de la secuencia de valores. Lo que `RandomAccessFile` proporciona son una serie de métodos que permiten modificar el valor de este apuntador, indicando de forma explícita su posición. Por tanto, para acceder a una posición cualquiera del fichero, primero es necesario un paso previo donde se indica la posición en la que se quiere operar, invocando alguno de estos métodos de posicionamiento del apuntador y, posteriormente, se puede llevar a cabo la lectura o la escritura.

Para gestionar el posicionamiento del apuntador, la clase define los siguientes métodos:

1. `void seek(long pos)`. Ubica el apuntador exactamente en la posición especificada por el parámetro `pos`, medido en bytes, de modo que cualquier acceso a los datos será justo sobre el siguiente byte. No hay ninguna restricción en el valor de este parámetro, siendo posible ubicar el apuntador mucho más allá del final real del archivo. Esto puede parecer un poco raro, pero pronto verá el motivo.

La figura 2.9 muestra un esquema del funcionamiento de este método dado un archivo orientado a bytes que contiene un conjunto de enteros. Recuerde que cada entero ocupa 4 bytes.

El siguiente código de ejemplo usa el método `seek` para mover el apuntador del archivo a diferentes posiciones de manera arbitraria, tal y como se podría hacer con el índice de un array.

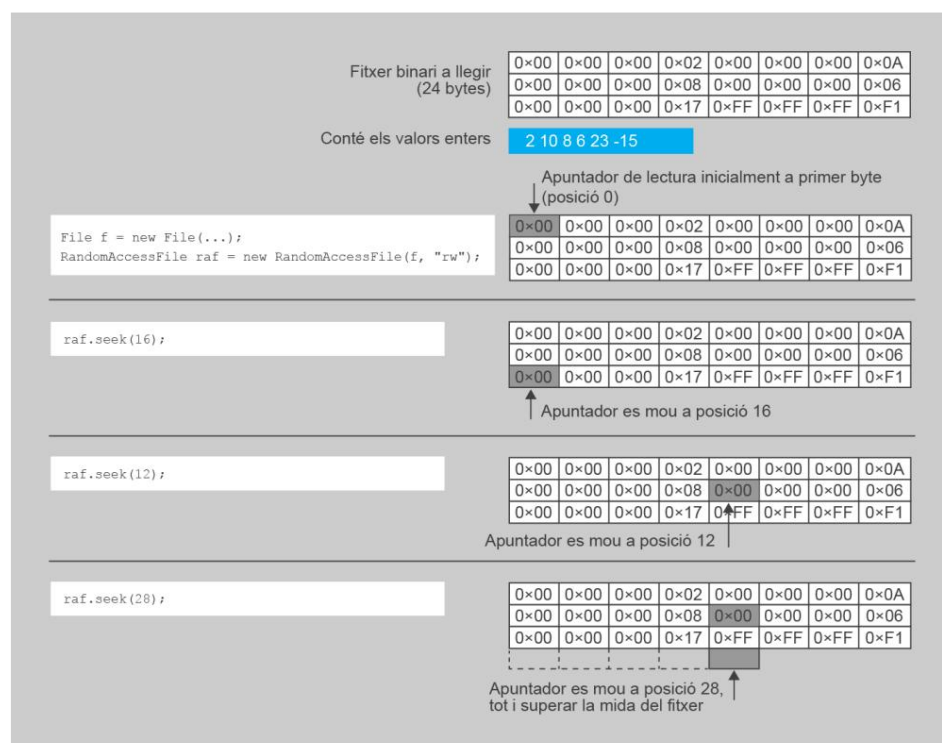
```
1 package unidad6.apartado2.ejemplos; 2 importe java.io.File; 3  
importe java.io.RandomAccessFile; 4  
public class MoverApuntadorSeek {
```

```

5 public static void main (String[] args) {
6     MoverApuntadorSeek programa = new MoverApuntadorSeek();
7     programa.inicio();
8 }
9 public void inicio() {
10     try {
11         File f = new File("Enters.bin");
12         RandomAccessFile raf = new RandomAccessFile(f, "r");
13         raf.seek(20);
14         long pos = raf.getFilePointer();
15         System.out.println("El apuntador está en la posición " raf.seek(0);          + pos);
16
17         pos = raf.getFilePointer();
18         System.out.println("El apuntador está en la posición " raf.seek(100);      + pos);
19
20         pos = raf.getFilePointer();
21         System.out.println("El apuntador está en la posición " raf.close();        + pos);
22
23     } catch (Exception e) {
24         //Excepción!
25         System.out.println("Error: "          + e);
26     }
27 }
28 }

```

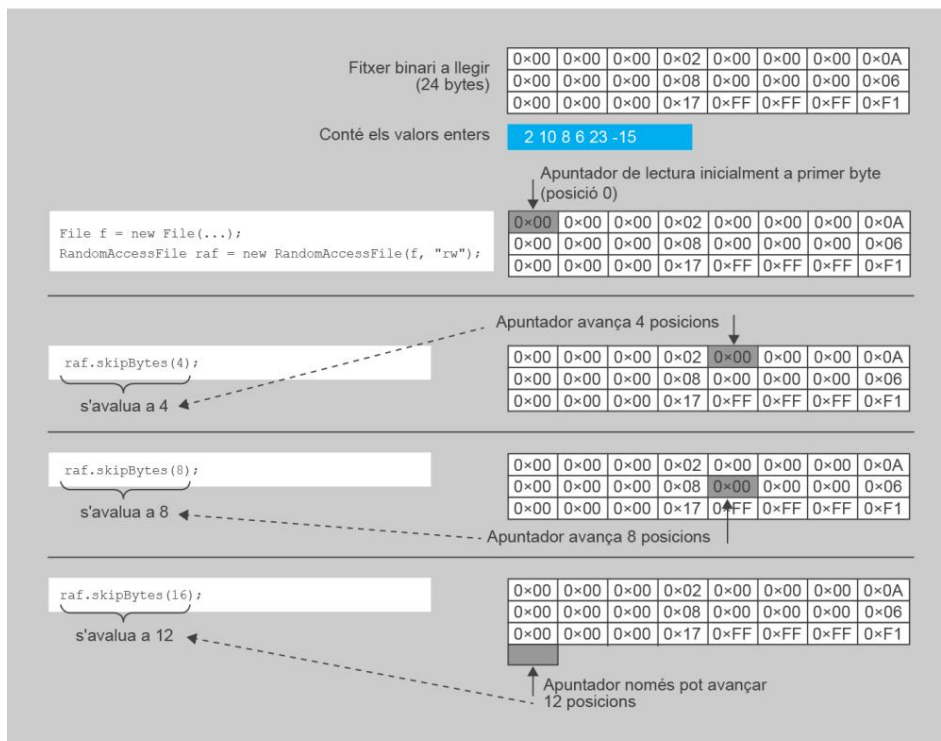
Figura 2. 9. Acceso aleatorio mediante posicionamiento de apuntador usando el método `seek`



2. `int skipBytes(int n)`. Avanza la posición del apuntador en `n` bytes, de modo que éste pasa a valer valor actual + `n`. Su invocación evalúa el número de bytes que realmente se ha adelantado. Hay que tener en cuenta que se puede dar el caso cuando el valor resultante no sea igual a `n`. Por ejemplo, si el apuntador llega al final del archivo, el desplazamiento del apuntador se detiene. Por tanto, Independientemente del valor del parámetro `n`, siempre debe controlar la posición del apuntador del archivo a partir del valor devuelto.

La figura 2.10 muestra un ejemplo de cómo se modifica el apuntador a partir de un seguido de llamadas al método `skipBytes`. Fíjese que si a la hora de ejecutarla el apuntador sobrepasa el tamaño del archivo, éste incrementa su tamaño con nuevos bytes, todos a 0.

Figura 2.10. Acceso aleatorio mediante posicionamiento de apuntador usando el método `skipBytes`



El siguiente código de ejemplo utiliza el método `skipBytes` para mover el apuntador a diferentes posiciones dentro del archivo. Fíjese cómo, en el último posicionamiento, se intenta ir mucho más allá del tamaño del archivo. Si se sobrepasa ese límite, el apuntador nunca avanza más allá del tamaño del archivo.

```
1 package unidad6.apartado2.ejemplos;
2 import java.io.File;
3 import java.io.RandomAccessFile;
4 public class MoverApuntadorSkipBytes {
5     public static void main (String[] args) {
6         MoverApuntadorSkipBytes programa = new MoverApuntadorSkipBytes();
7         programa.inicio();
8     }
9     public void inicio() {
10        try {
11            File f = new File("Enters.bin");
12            RandomAccessFile raf = new RandomAccessFile(f, "r");
13            long skip = raf.skipBytes(20);
14            long pos = raf.getFilePointer();
15            System.out.print("El apuntador ha avanzado + skip + " + " posiciones. ");
16            System.out.println("Está en la posición " + pos);
17            skip = raf.skipBytes(8);
18            pos = raf.getFilePointer();
19            System.out.print("El apuntador ha avanzado + skip + " + " posiciones. ");
20            System.out.println("Está en la posición " + pos);
21            //Se intenta avanzar mucho más allá del tamaño del archivo
22            skip = raf.skipBytes(400);
23            pos = raf.getFilePointer();
24            System.out.print("El apuntador ha avanzado + skip + " + " posiciones. ");
25            System.out.println("Está en la posición " + pos);
26            raf.close();
27        }
28    }
29 }
```

```
27     } catch (Exception e) {  
28         //Excepción!  
29         System.out.println("Error: " + e);  
30     }  
31 }  
32 }
```

2.4.2 Lectura de datos relativa

Mediante los métodos `seek` y `skipBytes` es posible indicar desde que posición del archivo, medida en bytes, se quiere llevar a cabo la lectura, la cual se lleva a término con las mismas instrucciones que el tratamiento secuencial (los métodos `readInt()`, `readDouble`, etc.), y de hecho, mientras no se vuelva a invocar ninguno de los dos métodos de posicionamiento del apuntador, el tratamiento del archivo pasa a ser totalmente secuencial. Cada vez que se lee un valor, el apuntador avanza tantos bytes como el tamaño del tipo del valor (4 bytes si es un entero, 8 si es un double, etc.).

El siguiente programa de ejemplo utiliza los métodos `seek` y `skipBytes` para leer los datos de un archivo que sólo contiene valores enteros. Se ignora la primera mitad de los valores y se van mostrando los valores de la segunda mitad de forma intermitente. O sea, si hay 20 valores, se saltan los 10 primeros valores y se muestran por pantalla los valores 11, 13, 15, etc. A lo largo de su ejecución, mediante el método `getFilePointer`, se va controlando la evolución del apuntador de lectura antes de acceder a cada valor.

```
1 package unidad6.apartado2.ejemplos;  
2 import java.io.File;  
3 import java.io.RandomAccessFile;  
4 public class LeerEnterosRelativo {  
5     public static void main (String[] args) {  
6         LeerEnterosRelativo programa = new LeerEnterosRelativo();  
7         programa.inicio();  
8     }  
9     public void inicio() {  
10         try {  
11             File f = new File("Enters.bin");  
12             RandomAccessFile raf = new RandomAccessFile(f, "r");  
13             long numEnters = f.length()/4;  
14             long mitad = numEnters/2;  
15             //Apuntador al inicio del entero en mitad del archivo  
16             raf.seek(mitad*4);  
17             long pos = raf.getFilePointer();  
18             //Se lee hasta llegar al final del archivo  
19             do {  
20                 System.out.print("(apuntador en la posición " + pos + ") ->");  
21                 //Se lee un entero  
22                 int valor = raf.readInt();  
23                 System.out.println(" Se ha leído el valor " + valor);  
24                 //Se salta el siguiente entero (4 bytes)  
25                 raf.skipBytes(4);  
26                 pos = raf.getFilePointer();  
27             } while (pos < f.length());  
28         } catch (Exception e) {  
29             //Excepción!  
30             System.out.println("Error leyendo datos: " + e);  
31         }  
32     }  
33 }
```

Como sucedía en el tratamiento secuencial, a la hora de realizar lecturas mediante acceso relativo hay que tener cuidado y usar siempre el método que corresponde al tipo del valor al que se quiere acceder, así como procurar no acceder a posiciones más allá del tamaño real del fichero.

Para el caso del acceso relativo, además, también debe ser muy cuidadoso para indicar cuál debe ser la posición del apuntador. Éste siempre debe estar en la primera posición del grupo de bytes que conforman cada valor. En caso contrario, el programa se comportará de forma errática, ya que se leerán bytes de diferentes valores mezclados. Por ejemplo, si un archivo sólo contiene valores enteros, las únicas posiciones válidas para realizar lecturas son la 0, 4, 8, 12, etc.

2.4.3 Escritura de datos relativa

La clase `RandomAccessFile` tiene la particularidad de que, siempre que realiza una escritura sobre un archivo que ya existe, sólo modifica el valor donde se ubica el apuntador y ningún otro. Por tanto, con la ayuda del método de posicionamiento es posible modificar los valores dentro de un archivo orientado a byte directamente, tal y como si fueran los valores dentro de un array. De hecho, normalmente, la escritura de datos relativa sólo tiene sentido cuando se trabaja sobre un archivo ya existente.

Es necesario calcular muy bien cuál es la posición en número de bytes donde debe ir el apuntador.

El acceso relativo a un archivo evita tener que cargar todos los datos a memoria, por ejemplo a un array, y luego volver a escribirlos si se quieren realizar modificaciones a valores individuales. Estas enmiendas se pueden realizar directamente sobre las posiciones del archivo.

Una vez que el apuntador se encuentra en la posición que se quiere modificar, el proceso de escritura mediante acceso relativo es similar al de lectura, simplemente cambiando la invocación de métodos de lectura por los de escritura (`writeInt(...)`, `writeDouble(...)`, etc.).

El siguiente programa muestra un ejemplo de escritura mediante acceso relativo, usando los métodos de posicionamiento del apuntador. En este caso, el programa accede a un archivo orientado a byte que contiene un conjunto de valores enteros y modifica los valores que ocupan una posición múltiple de cinco con su propia posición. O sea, reemplaza el quinto valor por un 5, el décimo elemento por un 10, el decimoquinto elemento por un 15, etc. hasta llegar al final del archivo. Al finalizar la escritura, se muestran los nuevos valores del archivo por pantalla.

```
1 package unidad6.apartado2.ejemplos; 2 importe java.io.File; 3 importe
java.io.RandomAccessFile; 4 public class
EscribirEnterosRelativo {

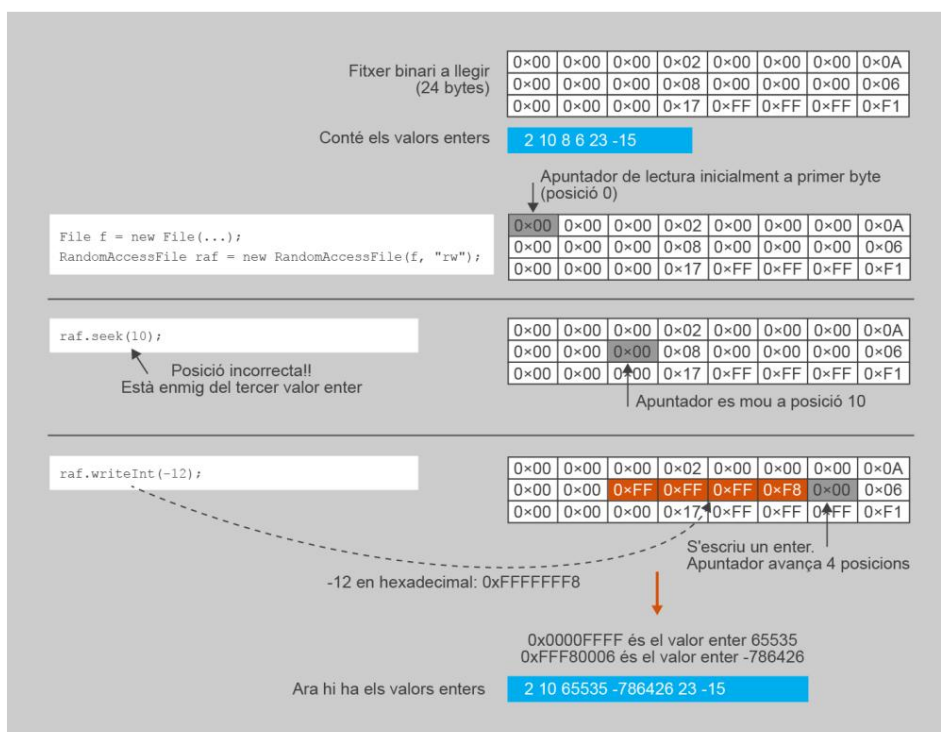
5     public static void main (String[] args) {
6         EscribirEnterosRelativo programa = new EscribirEnterosRelativo(); programa.inicio();
7
8
9     } public void inicio() { try {
10
```

```
11     File f = new File("Enters.bin");
12     //Se muestra el contenido original (si existe)
13     System.out.println("Valores iniciales del archivo.");
14     mostrarArchivoBinario(f);
15     //Se hacen las modificaciones
16     editarArchivoBinario(f);
17     //Y ahora se muestra el nuevo contenido
18     System.out.println("Nuevos valores del archivo.");
19     mostrarArchivoBinario(f);
20 } catch (Exception e) {
21     //Excepción!
22     System.out.println("Error escribiendo datos: " + e);
23 }
24 }
25 /** Muestra por pantalla todos los valores de un archivo orientado a byte que
26  * contiene enteros.
27  *
28  * @param f Ruta del archivo a mostrar
29  */
30 public void mostrarArchivoBinario(File f) {
31     try {
32         RandomAccessFile raf = new RandomAccessFile(f, "r");
33         long pos = raf.getFilePointer();
34         while (pos < f.length()) {
35             int valor = raf.readInt();
36             System.out.print(" " + valor);
37             pos = raf.getFilePointer();
38         }
39         raf.close();
40         System.out.println();
41     } catch (Exception e) {
42         //Excepción!
43         System.out.println("Error leyendo datos: " + e);
44     }
45 }
46 /** Modifica el contenido de un archivo orientado a byte que contiene enteros, de
47  * modo que cada 5 posiciones se sobrescribe el valor por el número de
48  * la propia posición.
49  *
50  * @param f Ruta del archivo a modificar
51  */
52 public void editarFicheroBinario(File f) {
53     try {
54         RandomAccessFile raf = new RandomAccessFile(f, "rw");
55         //Se avanza hasta el inicio del quinto entero
56         //valor 1 = posición 0, valor 2 = posición 4, valor 3 = posición 8
57         //valor 4 = posición 12, valor 5 = posición 16
58         raf.seek(16);
59         long pos = raf.getFilePointer();
60         int y = 1;
61         while (pos < f.length()) {
62             //Se escribe un valor en el apuntador actual
63             raf.writeInt(i*5);
64             i++;
65             //Se saltan 4 valores enteros. Estos no se tocan
66             raf.skipBytes(4*4);
67             //En total, el apuntador ha avanzado 5 valores (escrito + saltados)
68             pos = raf.getFilePointer();
69         }
70         //Escritura finalizada
71         raf.close();
72     } catch (Exception e) {
73         //Excepción!
74         System.out.println("Error escribiendo datos: " + e);
75     }
76 }
77 }
```

Como ocurriría con la lectura, hay que tener mucho cuidado a la hora de posicionar al apuntador, de modo que éste esté siempre en el primer byte del valor que se quiere sobrescribir. En este caso, sin embargo, hacer bien los cálculos es especialmente crítico, ya que de lo contrario echará a perder datos dentro del archivo.

La figura 2.11 muestra un ejemplo de escritura sobre una posición inválida al realizar un acceso relativo incorrecto sobre un archivo orientado a byte con 6 valores enteros. Fíjese que el apuntador se sitúa justo en mitad del tercer valor, de modo que, al escribir los 4 bytes de un nuevo valor entero, se sobrescribe la segunda mitad del tercer valor (2 bytes) y la primera mitad del cuarto (2 bytes más). El resultado es que se cambia parte de los bytes correspondientes a estos valores, por lo que ahora pasan a representar valores completamente distintos. Ya a simple vista, estos nuevos valores sólo dejan entrever que algo se ha hecho mal.

Figura 2.11. Escritura incorrecta en un acceso relativo.



Seek más allá del tamaño del archivo

En el proceso de escritura mediante acceso relativo también existe un caso especial. Se trata de cuando se ha llamado el método seek de modo que el apuntador se ubica más allá del final del archivo (por ejemplo, el archivo tiene 80 bytes y se ubica en la posición 100), y entonces se lleva a cabo una escritura. En este caso, el tamaño del archivo se incrementa automáticamente hasta la posición del apuntador, llenando todos los nuevos bytes con 0. Una vez hecho, entonces se materializa la escritura.

Por ejemplo, si dado un archivo de 80 bytes se usa el método seek para ubicar el apuntador hasta la posición 100 y entonces se hace la escritura de un entero, el archivo crecerá hasta los 100 bytes, y sus últimos 20 bytes estarán todos a 0. Entonces se llevará a cabo el final e bytes. El siguiente programa muestra este caso.

Mediante el sistema de acceso relativo, también es posible escribir datos directamente desde el final de un archivo existente, de forma que se anexen.

```
1 package unidad6.apartado2.ejemplos;
2 importe java.io.File;
3 importe java.io.RandomAccessFile;
4 public class SeekCreixFicheroBinario {
5     public static final String NOMBRE_FICHERO = "Seek.bin";
6     public static void main (String[] args) {
7         SeekCreixFicheroBinario programa = new SeekCreixFicheroBinario();
8         programa.inicio();
9     }
10    public void inicio() {
11        File f = new File(NOMBRE_FICHERO);
12        crearArchivo(f);
13        System.out.println("El tamaño inicial es " ejecutaSeek(f);  + f.length());
14
15        System.out.println("El tamaño a la hora de hacer seek y escribir es"      + f.length
16        ());
17    }
18    /** Crea un archivo cualquiera de 80 bytes.
19     *
20     * @param f Ruta del archivo a crear
21     */
22    public void crearArchivo(File f) {
23        try {
24            RandomAccessFile raf = new RandomAccessFile(f, "rw");
25            for (int i = 0; i < 20; i++) {
26                raf.writeInt(i);
27            }
28            raf.setLength(20*4);
29            raf.close();
30        } catch (Exception e) {
31            //Excepción!
32            System.out.println("Error escribiendo datos: "      + e);
33        }
34    }
35    /** Crea un archivo cualquiera de 80 bytes.
36     *
37     * @param f Ruta del archivo a crear
38     */
39    public void ejecutaSeek(File f) {
40        try {
41            RandomAccessFile raf = new RandomAccessFile(f, "rw");
42            //Se va 20 bytes más allá del tamaño del archivo
43            raf.seek(f.length() + 20);
44            //Se escribe un valor cualquiera
45            raf.writeInt(100);
46            raf.close();
47        } catch (Exception e) {
48            //Excepción!
49            System.out.println("Error escribiendo datos: "      + e);
50        }
51    }
```

Mediante este sistema, también es posible escribir datos directamente desde el final de un archivo existente, de modo que se anexasen.

Reto 5. Haga un programa que, dado un archivo orientado a byte que contiene cualquier número de valores reales, los ordene de menor a mayor. Esta tarea debe llevarla directamente sobre el archivo, y no cargando los datos a un array, ordenando y después escribiéndolos de nuevo en el archivo. Para ver que funciona, hacer que muestre por pantalla los valores contenidos antes y después de la ordenación.

2.5 Soluciones a los retos propuestos

Reto 1

```
1 package unidad6.apartado2.ejemplos; 2 importe java.io.File; 3 importe
java.util.Scanner; 4 public class LeerEnteros
{ public static final int NUM_VALORS = 15; public static
void main (String[] args) {
5
6
7     LeerEnters programa = new LeerEnters(); programa.inicio();
8
9 }
10 public void inicio() {
11     Scanner lector = null;
12     try {
13         //Se intenta abrir el archivo
14         File f = new File("Reals.txt");
15         lector = new Scanner(f);
16         //De momento, el máximo es el primer valor
17         double maxim = lector.nextDouble();
18         //Recorremos el resto de valores
19         for (int i = 1; i < NUM_VALORS; i++) {
20             double valor = lector.nextDouble();
21             //¿El nuevo valor es mayor?
22             if (maximo < valor) {
23                 maximo = valor;
24             }
25         }
26         System.out.println("El mayor valor es: " + maximo);
27     }
28     //Excepción!
29     System.out.println("Error: " + e);
30 }
31 //Con o sin error, hay que cerrar el archivo
32 lector.close();
33 }
34 }
```

Reto 2

```
1 package unidad6.apartado2.ejemplos;
2 importe java.io.File;
3 importe java.io.PrintStream;
4 public class EscribirEntersDobles {
5     public static final int NUM_VALORS = 20;
6     public static void main (String[] args) {
7         EscribirEntersDobles programa = new EscribirEntersDobles();
8         programa.inicio();
9     }
10    public void inicio() {
11        PrintStream escritor = null;
12        try {
13            //Se intenta abrir el archivo
14            File f = new File("Enters.txt");
15            escritor = new PrintStream(f);
16            //Se escribe el primer valor, que no necesita un espacio antes
17            int valor = 1;
18            escritor.print(valor);
19            //Se van generando el resto de valores y escribiendo
20            for (int i = 1; i < 20; i++) {
21                if (i%5 == 0) {
22                    //Cada 5 elementos, se escribe como delimitador un salto de línea
23                    escritor.print("\n");
24                } else {
25                    //Si no, se escribe como delimitador un espacio en blanco
26                    escritor.print(" ");
27                }
28                //Se calcula y escribe el nuevo valor
29                valor = valor*2;
30                escritor.print(valor);
31            }
32            System.out.println("Archivo escrito satisfactoriamente.");
33        } catch (Exception e) {
34            //Excepción!
35            System.out.println("Error: " + e);
36        }
37        //Con o sin error, hay que cerrar el archivo
38        escritor.close();
39    }
40 }
```

Reto 3

```

1 package unidad6.apartado2.retos;
2 importe java.io.File;
3 importe java.util.Scanner;
4 importe java.io.PrintStream;
5 public class GenerarFicheroNotas {
6     //la palabra fin sirve como marca de final de archivo
7     public static final String MARCA_FI = "fin";
8     public static final double MARCA_FI_NOTES = -1;
9     public static void main (String[] args) {
10         GenerarFicheroNotas programa = new GenerarFicheroNotas();
11         programa.inicio();
12     }
13     public void inicio() {
14         try {
15             File in = new File("NotesMarca.txt");
16             Scanner lector = new Scanner(in);
17             File out = new File ("NotaMitja.txt"); salida //Se genera el archivo de

18             PrintStream escritor = new PrintStream(out);
19             boolean leer = true;
20             //Solo es necesario cambiar las escrituras a pantalla para el archivo de salida
21             //O sea, donde pone "System.out" poner "escritor"
22             while (leer) {
23                 String nombre = lector.next();
24                 if (MARCA_FI.equals(nombre)) {
25                     leer = false;
26                 } else {
27                     String apellido = lector.next();
28                     escritor.print("Estudiante: " + apellido); + nombre + ""
29                     //Note cómo un Scanner se puede pasar como parámetro
30                     double media = leerNotes(lector);
31                     escritor.println(" - Media: " + media);
32                 }
33             }
34             lector.close(); //Hay que cerrar los archivos
35             escritor.close();
36             System.out.println("Archivo escrito satisfactoriamente.");
37         } catch (Exception e) //Excepción!
38         { System.out.println("Error leyendo estudiantes: " + e);
39         }
40     }
41     /** Dado un Scanner en un archivo cuando el apuntador es sobre el inicio
42     * de las notas, se extraen y se calcula la media.
43     * @param lector - Scanner a procesar
44     * @return Media de notas
45     */
46     public double leerNotes(Scanner lector) {
47         double nada = 0;
48         try {
49             //Ahora se lee hasta una marca de fin (-1)
50             boolean leer = true;
51             int numNotes = 0;
52             while (leer) {
53                 double valor = lector.nextDouble();
54                 if (valor == MARCA_FI_NOTES) { //Final
55                     leer = false;
56                 } else { nada //Se acumula el valor de las notas
57                     = nada + valor;
58                     numNotes++;
59                 }
60             }
61             nada = nada/numNotes; //Se calcula nota media
62         } catch (Exception e) //Excepción!
63         { System.out.println("Error leyendo notas: " + e);
64         }
65         return nada;
66     }
67 }

```

Reto 4

```
1 package unidad6.apartado2.retos;
2 importe java.io.File;
3 importe java.io.RandomAccessFile;
4 importe java.util.Arrays;
5 importe java.util.Random;
6 public class CalculsRealsBinario {
7     public static void main (String[] args) {
8         CalculsRealsBinario programa = new CalculsRealsBinario();
9         programa.inicio();
10    }
11    public void inicio() {
12        File f = new File("Reals.bin");
13        crearReales(f);
14        double[] valores = leerReals(f);
15        if (valores != null) {
16            //Todo correcto
17            Arrays.sort(valores);
18            for (int i = valores.length - 1; i >= 0; i--) {
19                System.out.println(valores[i]);
20            }
21        } else {
22            //Ha habido un error en la lectura
23            System.out.println("Error leyendo datos.");
24        }
25    }
26    /** Dado un archivo orientado a byte con reales, los lee todos
27     * y los pone en un array.
28     * @param f Ruta del archivo a leer
29     * @return Array de reales
30     */
31    public double[] leerReals(File f) {
32        try {
33            RandomAccessFile raf = new RandomAccessFile(f, "r");
34            //Cálculo del número de reales
35            long numReals = f.length() / 8;
36            //para inicializar un array es necesario un "int", no un "long"
37            int n = (int)numReals;
38            double[] array = new double[n];
39            for (int i = 0; i < numReals; i++) {
40                array[i] = raf.readDouble();
41            }
42            raf.close();
43            return array;
44        } catch (Exception e) {
45            //Excepción!
46            return null;
47        }
48    }
49    /** Genera un archivo orientado a byte con 20 valores reales, entre 0 y 100,
50     * al azar.
51     *
52     * @param f Ruta del archivo a generar
53     */
54    public void crearReals(File f) {
55        try {
56            RandomAccessFile raf = new RandomAccessFile(f, "rw");
57            Random r = new Random();
58            for (int i = 0; i < 20; i++) {
59                raf.writeDouble(100r.nextDouble());
60            }
61            raf.close();
62        } catch (Exception e) {
63            System.out.println("Error generando archivo: " + e);
64        }
65    }
66 }
```

Reto 5

```
1 package unidad6.apartado2.retos;
2 importe java.io.File;
3 importe java.io.RandomAccessFile;
4 public class OrdenarRealesBinario {
5     public static void main (String[] args) {
6         OrdenarRealsBinario programa = new OrdenarRealsBinario();
7         programa.inicio();
8     }
9     public void inicio() {
10         File f = new File ("Reals.bin");
11         System.out.println("Contenido original del archivo:");
12         mostrarArchivo(f);
13         ordenarArchivo(f);
14         System.out.println("-----");
15         System.out.println("Contenido ordenado del archivo:");
16         mostrarArchivo(f);
17     }
18     /** Dado un archivo orientado a byte que contiene reales, ordena directamente sobre
19      * el archivo todos sus valores, de menor a mayor.
20      *
21      * @param f Ruta al archivo con los datos a ordenar
22      */
23     public void ordenarArchivo(File f) {
24         try {
25             RandomAccessFile raf = new RandomAccessFile(f, "rw");
26             //Cada real ocupa 8 bytes, por tanto, hay que avanzar de 8 en 8
27             for(long i = 0; i < f.length(); i = i + 8) {
28                 for(long j = i + 8; j < f.length(); j = j + 8) {
29                     //Se lee el valor en la posición "i"
30                     raf.seek(i);
31                     double valorI = raf.readDouble();
32                     //Se lee el valor en la posición "j"
33                     raf.seek(j);
34                     double valorJ = raf.readDouble();
35                     //Se comparan
36                     if (valorI > valorJ) {
37                         //Si "i" mayor que "j", se intercambian el sitio
38                         raf.seek(i);
39                         raf.writeDouble(valorJ);
40                         raf.seek(j);
41                         raf.writeDouble(valorI);
42                     }
43                 }
44             }
45             raf.close();
46         } catch (Exception e) {
47             System.out.println("Error ordenando archivo: " + e);
48         }
49     }
50     /** Dado un archivo orientado a byte que contiene reales, muestra los valores
51      * por pantalla.
52      *
53      * @param f Ruta del archivo a mostrar
54      */
55     public void mostrarArchivo(File f) {
56         try {
57             RandomAccessFile raf = new RandomAccessFile(f, "r");
58             long numReals = f.length()/8;
59             for (int i = 0; i < numReals; i++) {
60                 System.out.println(raf.readDouble());
61             }
62             raf.close();
63         } catch (Exception e) {
64             System.out.println("Error mostrando archivo: " + e);
65         }
66     }
67 }
```


3. Tratamiento modular de datos. El juego de combates en la arena

Los principios de la modularidad también son aplicables para resolver problemas en los que se realizan operaciones con archivos. De hecho, normalmente dentro de las aplicaciones suelen generarse módulos claramente diferenciados vinculados a la gestión de todos los datos persistentes.

En este apartado se plantea un ejemplo de aplicación modular en el que se lleva a cabo el tratamiento de datos almacenados en ficheros. Concretamente, se estudia cómo ampliar una aplicación ya existente, generada usando los principios de modularidad, de modo que estos principios se sigan manteniendo en la nueva versión con nuevas funcionalidades.

3.1 Descripción del problema

El programa que sirve como punto de partida es un juego en el que el usuario, el jugador, se va enfrentando con sucesivos adversarios en una arena de combate. Tanto el jugador como sus adversarios son definidos por una serie de valores, que nombrarán sus atributos, mediante los cuales se indica su pericia luchando: resistencia, nivel de poder, capacidad de atacar o defender, etc.

Cada combate se divide en rondas, al inicio de las cuales el jugador y su adversario eligen secretamente una estrategia a seguir. En cada ronda se puede seguir una estrategia distinta. Según las estrategias elegidas por cada uno, el combate se irá resolviendo más favorablemente hacia uno u otro, hasta que finalmente se considere que uno de los dos ha sido derrotado. Si se derrota al adversario, se otorga una puntuación al jugador. Si el jugador es derrotado, termina la partida. El objetivo final del jugador es sobrevivir diez combates, logrando la máxima puntuación posible en el proceso.

En la sección "Anexos" de la web dispone del código fuente completo de la aplicación sobre la que se trabaja en este apartado. Estudie atentamente.

3.1.1 División del problema

Si bien el proceso de resolver las rondas de combate según los atributos de los luchadores tiene sus particularidades, para plantear cuál es el funcionamiento de la aplicación, se dejará en esta descripción general, sin entrar en mayor detalle. De todas formas, si desea hacer la idea más esclarecedora, a continuación se expone la descomposición del problema mediante diseño descendente:

1. Generar los atributos del nuevo jugador.
2. Anunciar inicio del combate.
 - (a) Mostrar estado del jugador.

3. Elegir al adversario.

4. Combatir.

(a) Mostrar estado de los luchadores.

i. Mostrar estado del jugador.

ii. Mostrar estado del adversario.

(b) Elegir estrategia del jugador.

(c) Elegir estrategia del adversario.

(d) Resolver resultados de estrategias.

i. Lanzar monedas.

ii. Penalizar a luchador.

iii. Dañar luchador.

iv. Curar luchador.

(e) Restaurar luchador.

5. Resolver resultado del combate.

(a) Otorgar puntuación.

(b) Subir de nivel.

(c) Finalización del juego.

3.1.2 Módulos del programa

Partiendo de la división en subproblemas del juego de combates de la arena, se ha realizado la división en módulos siguiente. Por un lado, en el package `joc.arena.reglas` estarán las clases:

- Monedas: para las tareas vinculadas al lanzamiento de monedas por resolver una ronda.
- Luchador: para las tareas vinculadas a la manipulación de los datos de un luchador (dañar, curar, etc.).
- Bestiario: para las tareas vinculadas a la generación de adversarios y el jugador.
- Combate: para las tareas vinculadas a la resolución de estrategias enfrentadas.

Por otra parte, en el package `joc.arena.interficie` se decide dividir las clases que tratan la pantalla y el teclado, por lo que habrá:

- EntradaTeclado: se encarga de las tareas importantes que son dadas por lo que escribe el usuario usando el teclado.
- SalidaPantalla: como la anterior, pero para mostrar información en pantalla.

La clase principal, `JocArena` está en el package que engloba los anteriores, `joc.arena`, dada la jerarquía de nombres.

Partiendo de esta descripción, se quieren añadir dos nuevas funcionalidades:

- Mantener un ranking con las diez mejores puntuaciones obtenidas. Éstas se guardan en un archivo, por lo que se mantienen entre partidas diferentes.
Al finalizar el juego, si un jugador ha obtenido una puntuación que merece estar entre las diez primeras, puede indicar cuáles son sus iniciales (tres letras), para que consten asociadas a la puntuación.
- Ahora mismo, los atributos de los adversarios están escritos en el código fuente del programa (en la clase `Bestiario`). Esto impide añadir nuevos adversarios fácilmente. Habría que modificar su código fuente y compilar de nuevo el programa.
Estaría bien que los datos de los adversarios se obtengan de un archivo, de modo que, sólo modificando este archivo, el programa ya incorpora siempre automáticamente a los adversarios contenidos.

3.2 Criterios de elección de tipos de archivo

Una de las primeras decisiones que deben tomarse cuando hay que tratar archivos es establecer cómo se almacenarán los datos (en formato carácter o byte) y el acceso que se llevará a cabo (secuencial o relativo). En muchos casos, cualquiera que sea el sistema elegido, el problema se podrá resolver igualmente. La única diferencia será la complejidad del algoritmo a implementar y el número de veces que debe leerse el archivo.
En cualquier caso, usar acceso relativo siempre implicará que el archivo será orientado a byte.

En la tabla 3.1 se resumen algunos pros y contras de cada tipo, para que los tenga en cuenta.

Tabla 3. 1. Ventajas y desventajas según el tipo de archivos

Tipo de archivo	Ventajas	Desventajas
Orientado a carácter	Son fáciles de crear y editar con herramientas externas (un editor de texto simple). Son fáciles de depurar (ver si su contenido es correcto). Al leerlos, es fácil comprobar mediante código de tipos de un dato datos antes de leerlo (métodos <code>hasNext...</code>).	El tamaño que ocupa cada valor puede ser muy variable. Es necesario controlar el número de valor en el archivo, o alguna marca de finalización Sólo permiten tratamiento secuencial. No se pueden sobrescribir fragmentos concretos. Es necesario reescribir todo el archivo al completo.
Orientado a byte	El tamaño que ocupa cada valor es siempre el mismo. Ligado al punto anterior, es fácil calcular cuántos valores contienen. Permiten el acceso relativo. Permiten sobrescrituras parciales de los datos.	El código es más complicado. No es fácil ver lo que contienen. No permiten comprobar los tipos de valores antes de leerlos. En caso de error, es difícil detectarlo.

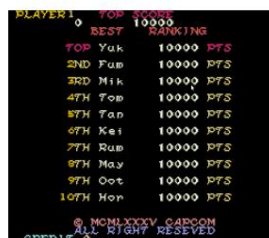
3.3 La biblioteca "juego.arena.ficheros"

Normalmente, el tratamiento de datos dentro de archivos se suele realizar en un package diferenciado. Esta decisión es coherente con la de disponer, por ejemplo, de otro vinculado a la entrada/salida de datos con el teclado y la pantalla. A la hora de elegir el nombre del package, es recomendable mantener también en su nombre la relación jerárquica entre módulos en la aplicación. Por tanto, este package se puede llamar `joc.arena.ficheros`.

Una aproximación sensata para enfocar la estructura de esta biblioteca es hacerlo de forma que cada archivo sea tratado por una clase diferente. En este caso, pues, serán necesarias dos clases nuevas, una para tratar el fichero con la lista de puntuaciones y otra para el tratamiento de los adversarios.

3.3.1 La clase Ranking

Esta clase será la encargada de gestionar el archivo con las máximas puntuaciones. Debe poder tanto leerlas para mostrarlas por pantalla como modificarlo para escribir nuevas.



PLAYER1	TOP SCORE
0	10000
BEST RANKING	
TOP Yuk	10000 PTS
2ND Fue	10000 PTS
3RD Mik	10000 PTS
4TH Top	10000 PTS
5TH Tan	10000 PTS
6TH Kei	10000 PTS
7TH Rue	10000 PTS
8TH May	10000 PTS
9TH Oct	10000 PTS
10TH Hon	10000 PTS

© MCMXXCV CARCON
ALL RIGHT RESEVED
CREDIT 0

Una lista de máximas puntuaciones.

De cara a tener una idea clara de cómo debe funcionar, es necesario decidir cuál será el funcionamiento exacto de la lista de puntuaciones máximas. Esta lista tendrá siempre 10 entradas, e inicialmente, cuando todavía no se ha jugado ninguna partida, habrá una serie de puntuaciones por defecto.

Elección de tipos de archivo y acceso

Antes de empezar es necesario escoger el nombre del archivo donde mantener los datos persistentes, su ubicación en el sistema de archivos, y su tipo y cómo tratarlo.

En cuanto al nombre, esto ya está a gusto del programador. En este caso, se puede llamar "ranking". Ahora bien, para elegir su ubicación, lo más sensato normalmente es que la ruta a un archivo sea relativa, para no imponer ninguna restricción sobre qué carpetas deben existir en cada ordenador donde se copie el programa. En ese caso se puede usar la propia carpeta de trabajo de la aplicación.

Sólo queda decidir el tipo de archivo. De entrada, siempre vale la pena prever la opción de un archivo de texto secuencial, ya que es el más sencillo de procesar y depurar.

En este sentido, un listado de 10 puntuaciones es una información que se puede tratar secuencialmente de manera simple, tanto para mostrarla por pantalla como para buscar dónde es necesario añadir nuevos elementos. La única parte en la que no todo encaja perfectamente es para insertar puntuaciones. Habrá que reescribir el archivo desde cero. Pero como es un archivo muy pequeño, 10 líneas, no es un gran problema y se considera asumible.

Propagación de errores

Cuando se realizan operaciones sobre archivo, pueden suceder una serie de situaciones erróneas. Por ejemplo, intentar leer datos de un archivo inexistente, leer datos cuando ya se ha llegado al final del archivo o de un tipo inesperado, al tener el archivo un formato incorrecto. En Java, estos errores se llaman excepciones, y es necesario controlarlos usando una sentencia try/catch.

En el caso de un programa modular con archivos, es muy importante que, cada vez que se invoca un método donde dentro de su código se tratan archivos, el código que ha llevado a cabo la invocación pueda establecer si todo ha ido bien o no. Por tanto, los métodos que tratan datos en ficheros deben poder avisar si han realizado todas las tareas correctamente o no.

La propagación de errores es el mecanismo mediante el cual un método avisa al código que le ha invocado si ha podido realizar su tarea correctamente o no.

Existen distintos mecanismos para llevar a cabo la propagación de errores. Para esta clase se ha usado lo más simple, que es reservar uno de los valores del parámetro de salida para indicar que ha sucedido un error (a menudo se utiliza el valor -1). Una vez evaluado el método invocado, comprobar si el resultado es el valor reservado o no. Si es así, es que ha habido una excepción dentro del método, y por tanto éste no ha podido llevar a cabo su tarea correctamente. Entonces, es necesario actuar en consecuencia.

Por ejemplo, mostrando un mensaje de error por pantalla.

En el caso de métodos que no disponen de ningún parámetro de salida (tipo void), se fuerza a que tengan un parámetro de salida de tipo entero, que servirá exclusivamente para establecer si todo ha ido bien o no. Normalmente se hace que se evalúen a 0 si todo ha ido bien y -1 si ha habido alguna excepción.

Por el momento, en el código de la clase Ranking sólo se puede ver la propagación de errores en cuanto a la parte de tratamiento de excepciones y retorno del valor -1.

Más adelante, cuando se vea cómo se invocan estos métodos desde otras clases, le quedará más claro cómo se controla la propagación para establecer si el método ha funcionado correctamente o no y actuar en consecuencia.

Código fuente de la clase

Dadas las necesidades expuestas, el código fuente de la clase puede ser el que se muestra a continuación. Entre sus particularidades, aparte de los aspectos anteriores, se encuentran los llamamientos internos al método generar Archivoinicial, que comprueba si el archivo de puntuaciones existe, y en su defecto, lo crea desde cero. Esto es más eficiente que decir que existe un error y no poder hacer nada. Sobre todo en caso de que se acaba de jugar una partida donde se ha obtenido una máxima puntuación, ¡ya que esta se perdería! Siempre es mejor corregir un error si cabe, en lugar de simplemente anunciarlo y no hacer nada más.

También, dadas las particularidades de los archivos secuenciales, observe en el método

entrarPuntuación como debe generarse un archivo temporal, ya que no se puede sobrescribir directamente el original.

```
1 package joc.arena.ficheros;
2 importe java.io.File;
3 importe java.io.PrintStream;
4 importe java.util.Scanner;
5 public class Ranking {
6     //Nombre archivo como constante
7     public static final String RANQUING = "Ranking";
8     /** Crea el archivo de puntuaciones inicial
9      * @return 0 si todo correcto, -1 si error
10     */
11     public int generarArchivolnicial() {
12         try {
13             File ranking = new File(RANQUING);
14             if (ranking.isFile() == false) {
15                 PrintStream ps = new PrintStream(ranking);
16                 for (int i = 0; i < 10; i++) {
17                     ps.println("IOC " + (10 - i)*10);
18                 }
19                 ps.close();
20             }
21             return 0;
22         } catch (Exception e) {
23             //Propagación de error
24             return -1;
25         }
26     }
27     /** Dada una puntuación, establece su posición en el archivo
28     * @param puntos Puntos a comprobar
29     * @return Posición para la puntuación. -1 si error.
30     */
31     public int buscarRanking(int puntos) {
32         try {
33             int err = generarArchivolnicial();
34             if (err == -1) {
35                 return -1;
36             }
37             File ranking = new File(RANQUING);
38             Scanner lector = new Scanner(ranking);
39             int pos = 0;
40             while (pos < 10) {
41                 lector.next();
42                 if (lector.hasNextInt()) {
43                     int ranqPts = lector.nextInt();
44                     if (puntos > ranqPts) {
45                         return pos;
46                     }
47                 } else {
48                     //Error en el formato del archivo
49                     return -1;
50                 }
51                 pos++;
52             }
53             lector.close();
54             return pos;
55         } catch (Exception e) {
56             return -1;
57         }
58     }
59     /** Inserta una puntuación en el ranking
60     * @param iniciales Iniciales del jugador
61     * @param puntos Puntuación alcanzada
62     * @param pos Posición en el ranking
63     * @return 0 si todo correcto, -1 si error.
64     */
65     public int entrarPuntuacio(String iniciales, int puntos, int pos) {
66         try {
67             int err = generarArchivolnicial();
```

```

68     if (err == -1) {
69         return -1;
70     }
71     File ranking = new File(RANQUING);
72     Scanner lector = new Scanner(ranking);
73     File tmp = new File (RANQUING + ".tmp");
74     PrintStream ps = new PrintStream(tmp);
75     //Reescribir anteriores a posición
76     for(int i = 0; y < pos; i++) {
77         String txt = lector.nextLine();
78         ps.println(txt);
79     }
80     //Escribir nueva puntuación
81     ps.println(iniciales + + puntos);
82     //Reescribir posteriores a posición
83     for(int i = pos + 1; y < 10; i++) {
84         String txt = lector.nextLine();
85         ps.println(txt);
86     }
87     ps.close();
88     lector.close();
89     //Se borra archivo antiguo y se pone el nuevo
90     ranking.delete();
91     tmp.renameTo(ranking);
92     return 0;
93 } catch (Exception e) {
94     return -1;
95 }
96 }
97 /** Lee las puntuaciones y las formatea como una cadena de texto
98  * @return Cadena de texto resultante. null si hay error
99  */
100 public String leerRanquing() {
101     try {
102         int err = generarArchivolnicial();
103         if (err == -1) {
104             return null;
105         }
106         String txtRanking = "Ranking de puntuaciones\n-----\n";
107         File ranking = new File(RANQUING);
108         Scanner lector = new Scanner(ranking);
109         for (int i = 0; y < 10; i++) {
110             //Leer iniciales
111             txtRanquing = txtRanquing + lector.next();
112             //Leer puntos
113             if (lector.hasNextInt()) {
114                 txtRanquing = txtRanquing + "\t" + lector.nextInt() + "\n";
115             } else {
116                 //Error en el formato del archivo
117                 return null;
118             }
119         }
120         lector.close();
121         return txtRanking;
122     } catch (Exception e) {
123         return null;
124     }
125 }
126 }

```

3.3.2 La clase Bestiario

La clase Bestiario es la encargada de generar los luchadores, tanto el jugador al inicio del juego como los sucesivos adversarios. En la versión original todo está

almacenado en el código fuente del programa. Ahora se quiere hacer una nueva versión, perteneciente a un package diferente, que obtenga toda la información desde un archivo. De esta forma, es posible usar diferentes listas de adversarios, o añadir de nuevos, sin tener que modificar el código fuente del programa. Sólo hace falta cambiar este archivo.

Elección de tipos de archivo y acceso

En este caso, el nombre del archivo donde tener guardados los datos de los adversarios será Adversarios, y también se usará una ruta relativa.

Antes de poder escribir el código fuente hay que tener bien claro cuál será el formato del archivo y su estructura. En este caso, se encuentra que debe adaptar una clase en la que todo se estructura dentro de un array originalmente, y lo que se quiere es trabajar sobre un archivo. El tipo de archivo que se parece más a un array en su modo de trabajar, para poder acceder a sus posiciones de manera directa, es un archivo de acceso relativo. Por tanto, se usará este sistema de tratamiento de datos, cosa que implica que es necesario usar un archivo orientado a byte.

Es necesario definir detalladamente la estructura del archivo y cómo se agrupan los conjuntos de valores almacenados (los atributos de los adversarios). Lo sensato es proponer una adaptación más o menos directa del array usado en la versión original. Ahora bien, para poder trabajar con un archivo orientado a byte de manera relativa, como si fuera un array, es importante que cada conjunto de datos tenga un tamaño fijo (en este caso, cada adversario). Esto significa que hay que acotar el tamaño del nombre. No puede ser de cualquier longitud. Para este programa, el tamaño se acotará a 15 caracteres. En el caso de nombres con menos de 15 caracteres, el resto de bytes se llenan todos a 0 hasta llegar a los 15 caracteres (30 bytes).

De esta forma, cada adversario ocupa siempre 50 bytes: 15*2 (nombre, compuesto de 15 caracteres de 2 bytes) + 5*4 (los 5 atributos, que son enteros de 4 bytes). La tabla 3.2 muestra un resumen de la estructura.

Tabla 3. 2. Estructura de los datos asociados a un adversario

Bytes 0-29	Bytes 30-33	Bytes 34-37	Bytes 38-41	Bytes 42-45	Bytes 46-49
Nombre (caracteres)	Nivel (entero)	Puntos (entero)	Vida Máx (entero)	Ataque (entero)	Defensa (entero)

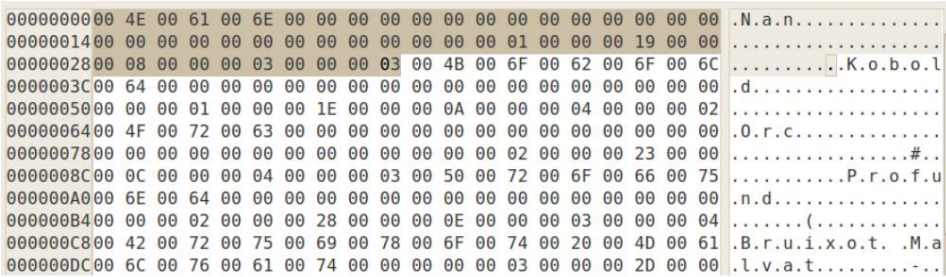
Dada esta estructura, la figura 3.1 mostraría un ejemplo de visualización del archivo mediante un editor hexadecimal. Los datos relativos al primer adversario (de nombre “Nan”) están sombreadas. Fíjese cómo los primeros 34 bytes corresponden en el nombre. Los 6 primeros son los caracteres del nombre propiamente y el resto están a 0. A partir del byte número 30, hay cinco enteros, cada uno de 4 bytes, asociados a sus atributos, por lo que:

- Nivel = 0x00000001
- Puntos = 0x00000019 (25 en decimal)
- Vida = 0x00000008

Con un archivo relativo orientado en byte, incluso es posible plantear búsquedas dicotómicas directamente sobre éste.

- Ataque = 0x00000003
- Defensa = 0x00000003

F igura 3. 1. Contenido del archivo de adversarios visualizado con un editor hexadecimal



Código fuente de la clase

Este caso es algo distinto del ranking de puntuaciones, ya que no se trata de añadir un módulo nuevo. Se trata de reemplazar un módulo con unas funciones concretas (gestionar adversarios a partir de unas variables globales) por uno con otras (que esta gestión se haga usando un archivo). Esto es una situación muy interesante, ya que si se aplican los principios de modularidad correctamente, las modificaciones en el resto del programa deberían ser mínimas.

Éste es, precisamente, el sentido de la modularidad. Poder cambiar un módulo (en este caso, una clase), por otro sin tener que tocar nada más. Tal y como se haría cuando se cambia una cajonera modular o un módulo de memoria del ordenador. Saque el antiguo, ponga el nuevo, y todo sigue funcionando sin tener que tocar nada más. Si se logra esto, todo se ha hecho perfectamente.

La estrategia para alcanzar este objetivo siempre es la misma. Intentar mantener los mismos métodos, con los mismos parámetros, que existían en la clase original, y sólo modificar el bloque de código que contienen. De este modo, no es necesario enmendar el código donde existen invocaciones en cualquiera de los métodos originales, ya que su definición será la misma en la clase nueva que en la antigua. Si es necesario, se pueden añadir nuevos métodos, pero es imprescindible mantener el formato de los que ya existían antes del cambio.

Por tanto, fíjese cómo, en la nueva clase, los métodos existentes son exactamente los mismos que había inicialmente, manteniendo su formato (nombre y parámetros). Sin embargo, se han creado algunos métodos nuevos para reutilizar código (por ejemplo, crear Adversario o leerNombre).



Cambiar un módulo de un programa debería ser cómo cambiar un mueble modular. Fuente: Bercik

```
1 package joc.arena.ficheros; 2 importe java.io.File; 3 importe
java.io.RandomAccessFile; 4 importe
java.util.Random; 5 importe juego.arena.reglas.Luchador; 6 public class
Bestiario {

7     //Constante con el nombre del archivo de adversarios private static
8     final File ADVERSARIOS = new File("Adversarios");
9     //Jugador: ID = 0 private
10    int[] jugador = {0, 1, 0, 10, 10, 3, 3, 3}; private Luchador luchador = new Luchador();
11
12    //Todos los métodos mantienen su declaración (nombre y parámetros)
```

```
13  /** Genera un nuevo jugador
14  *
15  * @return Un array con los datos de un jugador inicial
16  */
17  public int[] generarJugador() {
18      luchador.renovar(jugador);
19      return jugador;
20  }
21  /** Dado un nombre, genera el adversario correspondiente. Si ese nombre no existe,
22  * se genera al azar.
23  *
24  * @param nombreAdv Nombre del adversario a obtener
25  * @return El Luchador con este nombre, o null si no existe
26  */
27  public int[] buscarAdversario(String nombreAdv) {
28      try {
29          int[] adversario = null;
30          RandomAccessFile raf = new RandomAccessFile(ADVERSARIOS, "r");
31          long numAdv = ADVERSARIOS.length()/50;
32          for (int i = 0; i < numAdv; i++) {
33              raf.seek(50*i);
34              String nombre = leerNombre(raf);
35              if (nombre.equalsIgnoreCase(nombreAdv)) {
36                  adversario = crearAdversario(raf,i);
37              }
38          }
39          raf.close();
40          return adversario;
41      } catch (Exception e) {
42          return null;
43      }
44  }
45  /** Obtiene el adversario que hay en un orden concreto del archivo,
46  * en format array.
47  *
48  * @param raf Archivo relativo de dónde leerlo
49  * @param pos Posición del adversario dentro del archivo
50  * @return Luchador leído
51  */
52  public int[] crearAdversario(RandomAccessFile raf, int pos) {
53      try {
54          int[] adversario = new int[9];
55          raf.seek(pos*50 + 30);
56          adversario[0] = pos + 1;
57          adversario[1] = raf.readInt();
58          adversario[2] = raf.readInt();
59          adversario[3] = raf.readInt();
60          adversario[4] = adversario[3];
61          adversario[5] = raf.readInt();
62          adversario[6] = adversario[5];
63          adversario[7] = raf.readInt();
64          adversario[8] = adversario[7];
65          return adversario;
66      } catch (Exception e) {
67          return null;
68      }
69  }
70  /** Dado un archivo relativo orientado a byte, correctamente posicionado,
71  * lee el nombre del adversario.
72  *
73  * @param raf Archivo a tratar
74  * @return Nombre leído, o null si hay error
75  */
76  public String leerNombre(RandomAccessFile raf) {
77      try {
78          String nombre = "";
79          char c = raf.readChar();
80          while (c != 0x0000) {
81              nombre = nombre + c;
82              c = raf.readChar();
83          }
84      } catch (Exception e) {
85          return null;
86      }
87  }
```

```
83     }
84     return nombre;
85 } catch (Exception e) {
86     return null;
87 }
88 }
89 /**Dado un nivel, genera el adversario correspondiente al azar. Se trata
90  * de un adversario que sea al menos de este nivel
91  *
92  * @param nivel Nivel cercano al del adversario a obtener
93  * @return Un adversario
94  */
95 public int[] elegirAdversarioAzar(int nivel) {
96     try {
97         RandomAccessFile raf = new RandomAccessFile(ADVERSARIOS,"r");
98         Random rnd = new Random();
99         int numAdv = (int)raf.length()/50;
100         int[] adversario = null;
101         boolean buscar = true;
102         while (buscar) {
103             int y = rnd.nextInt(numAdv);
104             adversario = crearAdversario(raf, y);
105             int nivelAdv = luchador.leerNivel(adversario);
106             int dif = nivel - nivelAdv;
107             if ((dif >= -1)&&(dif <= 1)) {
108                 buscar = false;
109             }
110         }
111         raf.close();
112         return adversario;
113     } catch (Exception e) {
114         return null;
115     }
116 }
117 /** Transforma un identificador de Luchador a su nombre.
118  *
119  * @param id Identificador
120  * @return La cadena de texto con el nombre.
121  */
122 public String traducirIDANom(int id) {
123     if (id == 0) {
124         return "Aventurero";
125     }
126     id--;
127     try {
128         RandomAccessFile raf = new RandomAccessFile(ADVERSARIOS, "r");
129         int pos = 50*id;
130         String nombre = "DESCONOCIDO";
131         if (pos < raf.length()) {
132             raf.seek(pos);
133             nombre = leerNombre(raf);
134         }
135         raf.close();
136         return nombre;
137     } catch (Exception e) {
138         return "DESCONOCIDO";
139     }
140 }
141 /** Dice si existe el archivo de adversarios
142  *
143  * @return Si existe (true) o no (false)
144  */
145 public boolean existeArchivo() {
146     return ADVERSARIOS.isFile();
147 }
148 }
```

3.3.3 La clase auxiliar EditorBestiari

Herramientas de desarrollo

de juegos

Es muy habitual que los programadores de videojuegos tengan herramientas auxiliares por generar los archivos de datos de juegos que están creando. Éstas se llaman [herramientas de desarrollo de juegos](#).

El problema de los archivos orientados a byte es que no resultan fáciles de editar. Por eso, para este caso, puede resultar útil disponer de un programa auxiliar que sirva de editor de la lista de adversarios, de modo que sea fácil manipularla. Éste puede visualizar, añadir y borrar entradas en el archivo binario.

Código fuente de la clase

A continuación se muestra el código fuente de este programa auxiliar, creado monolíticamente en una sola clase, pero aplicando diseño descendente. El archivo de adversarios se puede editar partiendo desde cero, o bien desde uno ya existente. Para simplificar su código, todas las operaciones de añadir o eliminar trabajan sobre el final del archivo del archivo.

```
1 package joc.arena.ficheros;
2 importe java.io.File;
3 importe java.io.RandomAccessFile;
4 importe java.util.Scanner;
5 public class EditorBestiari {
6     public static final int MAX_CHAR_NOM = 15;
7     public static final int MEDIDA_BYTES_ADV = 50;
8     public static void main (String[] args) {
9         EditorBestiario programa = new EditorBestiari();
10        programa.inicio();
11    }
12    public void inicio() {
13        Archivo archivo = preguntarArchivo();
14        boolean ejecutar = true;
15        while (ejecutar) {
16            ejecutar = tratarMenu(fichero);
17        }
18    }
19    /** Pregunta al usuario el nombre del archivo a editar.
20     * @return Ruta al archivo a editar.
21     */
22    public File preguntarArchivo() {
23        Scanner lector = new Scanner(System.in);
24        System.out.print("Nombre del archivo a editar: ");
25        String nombre = lector.nextLine();
26        File archivo = new File(nombre);
27        return archivo;
28    }
29    /** Realiza el tratamiento del menú del usuario.
30     * @param archivo Archivo a tratar
31     * @return Si es necesario seguir ejecutando del programa (true) o aún no (false)
32     */
33    public boolean tratarMenu(File archivo) {
34        mostrarArchivo(archivo);
35        System.out.println("-----");
36        System.out.println("[A]añadir\t[E]liminar último\t[S]salir");
37        boolean preguntar = true;
38        Scanner lector = new Scanner(System.in);
39        while (preguntar) {
40            System.out.print("Accio: ");
41            String respuesta = lector.nextLine();
42            if ("A".equalsIgnoreCase(respuesta)) {
43                añadirAdversario(fichero);
44                preguntar = false;
45            } else if ("E".equalsIgnoreCase(respuesta)) {
46                eliminarAdversario(fichero);
47                preguntar = false;
```

```

48     } else if ("S".equalsIgnoreCase(respuesta)) {
49         return false;
50     } else {
51         System.out.print("Acción incorrecta...");
52     }
53 }
54 return true;
55 }
56 /** Muestra el contenido del archivo por pantalla
57  * @param archivo Ruta al archivo a mostrar
58  */
59 public void mostrarArchivo(File archivo) {
60     try {
61         if (archivo.isFile() == false) {
62             System.out.println("Aún no se ha creado el archivo.");
63         } else {
64             RandomAccessFile raf = new RandomAccessFile(archivo, "r");
65             long numAdversaris = archivo.length()/MIDA_BYTES_ADV;
66             if (numAdversarios == 0) {
67                 System.out.println("El archivo está vacío.");
68             } else {
69                 for (int i = 0; i < numAdversaris; i++) {
70                     String nombre = leerNombre(raf);
71                     System.out.print(nombre);
72                     for (int z = 0; z < (MAX_CHAR_NOM - nombre.length()); z++) {
73                         System.out.print(" ");
74                     }
75                     //Cada adversario ocupa 50 bytes. El pedazo del nombre ocupa 15*2
76                     raf.seek(i*MIDA_BYTES_ADV + MAX_CHAR_NOMBRE*2);
77                     System.out.print("\tNivel: " + raf.readInt());
78                     System.out.print(" (puntos: " + raf.readInt() + ")");
79                     System.out.print("\tVIDA: " + raf.readInt());
80                     System.out.print("\tATAC: " + raf.readInt());
81                     System.out.println("\tDEFENSA: " + raf.readInt());
82                 }
83             }
84             raf.close();
85         }
86     } catch (Exception e) {
87         System.out.println("Error accediendo al archivo!");
88     }
89 }
90 /** Dado un archivo relativo orientado a byte, correctamente posicionado,
91  * lee el nombre del adversario.
92  * @param raf Archivo a tratar
93  * @return Nombre leído, o null si hay error
94  */
95 public String leerNombre(RandomAccessFile raf) {
96     try {
97         String nombre = "";
98         char c = raf.readChar();
99         while (c != 0x0000) {
100             nombre = nombre + c;
101             c = raf.readChar();
102         }
103         return nombre;
104     } catch (Exception e) {
105         return null;
106     }
107 }
108 /** Escribe un nuevo adversario al final del archivo, preguntando todo lo necesario
109  * al usuario.
110  * @param archivo Ruta al archivo al que hay que añadir el adversario.
111  */
112 public void añadirAdversario(File archivo) {
113     try {
114         RandomAccessFile raf = new RandomAccessFile(archivo, "rw");
115         //Se pone al final de todo
116         raf.seek(archivo.length());
117         System.out.println("Escribe el nombre del adversario (máx. 12 letras: ");

```

```
118 Scanner lector = new Scanner(System.in);
119 String nombre = lector.nextLine();
120 //Escribir el nombre (máx. 15 caracteres)
121 int err = escribirNombre(nombre, raf);
122 if (err == -1) {
123     System.out.println("Error escribiendo datos en el archivo " + archivo);
124
125     //Escribir valores - Nivel:XP:PV:Max PV:Ataque:Max Ataque:max Defensa
126     System.out.print("Escribe sus atributos, separados por espacios. ");
127     System.out.println("(5 enteros = Nivel Puntos PV Ataque Defensa:");
128     int[] valores = leerValores(lector);
129     for (int i = 0; i < valores.length; i++) {
130         raf.writeInt(valores[i]);
131     }
132 }
133 raf.close();
134 } catch (Exception e) {
135     System.out.println("Error escribiendo datos en el archivo " + archivo);
136 }
137 }
138 /** Lee cinco valores de tipo entero desde el teclado.
139  * @param lector Scanner que lee del teclado
140  * @return Los cinco valores leídos, ordenadamente
141  */
142 public int[] leerValores(Scanner lector) {
143     int[] valores = new int[5];
144     boolean preguntar = true;
145     while(preguntar) {
146         int numLeídos = 0;
147         for (int i = 0; i < valores.length; i++) {
148             if (lector.hasNextInt()) {
149                 valores[i] = lector.nextInt();
150                 numLeídos++;
151             } else {
152                 lector.next();
153             }
154         }
155         if (numLeídos == 5) {
156             preguntar = false;
157         } else {
158             System.out.println("Los 5 valores no han sido correctos.");
159         }
160     }
161     lector.nextLine();
162     return valores;
163 }
164 /** Dado un nombre en forma de cadena de texto, lo escribe a un archivo orientado a
165  * byte. Como máximo, el nombre puede tener 15 caracteres. El resto, se llena a 0.
166  * @param nombre Nombre a escribir
167  * @param raf Archivo relativo correctamente posicionado para la escritura
168  * @return Si ha funcionado (0) o no (-1)
169  */
170 public int escribirNombre(String nombre, RandomAccessFile raf) {
171     try {
172         int numChars = nombre.length();
173         if (numChars > MAX_CHAR_NOM) {
174             numChars = MAX_CHAR_NOM;
175         }
176         for (int i = 0; i < numChars; i++) {
177             raf.writeChar(nombre.charAt(i));
178         }
179         char blank = '\0';
180         for (int i = 0; i < (MAX_CHAR_NOM - numChars); i++) {
181             raf.writeChar(blank);
182         }
183         return 0;
184     } catch (Exception e) {
185         return -1;
186     }
187 }
```

```

188  /** Elimina el último adversario en un archivo.
189      * @param archivo Ruta del archivo a modificar
190      */
191  public void eliminarAdversario(File archivo) {
192      try {
193          RandomAccessFile raf = new RandomAccessFile(archivo, "rw");
194          long novaMida = archivo.length() - MIDA_BYTES_ADV;
195          //Eliminar el último adversario es borrar los últimos 50 bytes...
196          if (nuevaMida >= 0) {
197              raf.setLength(archivo.length() - MEDIDA_BYTES_ADV);
198          }
199          raf.close();
200      } catch (Exception e) {
201          System.out.println("Error borrando datos en el archivo " + archivo);
202      }
203  }
204 }

```

3.4 Enmiendas a los módulos originales

Una vez se han generado los nuevos módulos que amplían el programa original de forma que algunas de sus tareas dependan de la lectura o escritura de archivos, es necesario modificar el código original para que se usen los nuevos módulos. Una correcta aplicación de la modularidad debería minimizar el número de cambios en el código y hacer que éstos estén muy bien localizados. De hecho, el caso ideal sería que sólo es necesario añadir nuevo código, pero no modificar, o incluso eliminarlo, código ya existente.

3.4.1 En la clase EntradaTeclado

Nótese que el cambio de la clase Bestiario no afecta en absoluto a esta clase, todo y que la utiliza. Esto significa que se ha aplicado correctamente la modularidad.

```

1 //Ahora hay que importar la nueva versión
2  importe juego.arena.ficheros.Bestiario;
3  //Nuevo método
4  /** Pregunta las iniciales del jugador si alcanza una máxima puntuación.
5      * @return Iniciales (cadena de texto con 3 caracteres)
6      */
7  public String preguntarIniciales() {
8      Scanner lector = new Scanner(System.in);
9      System.out.println("¡has alcanzado una máxima puntuación!!");
10     String iniciales = "";
11     boolean leer = true;
12     while (leer) {
13         System.out.print("Escribe tus iniciales: ");
14         iniciales = lector.nextLine();
15         //Este método elimina espacios laterales
16         iniciales = iniciales.trim();
17         if (iniciales.length() == 3) {
18             leer = false;
19         } else {
20             System.out.print("Entrada incorrecta. ");
21         }
22     }
23     return iniciales.toUpperCase(); //Siempre se pondrán en mayúscula
24 }

```

El método "trim" elimina los espacios a derecha y izquierda de un texto.

La introducción de una máxima puntuación en el ranking implica poder pedir al usuario, si se cumplen las condiciones necesarias, que introduzca las sus iniciales. Si se quiere seguir estrictamente el planteamiento modular de la aplicación, esto quiere decir que es necesario un nuevo método en la clase EntradaTeclado. Éste debe comprobar que, efectivamente, se trata de unas iniciales (en este caso, se considera que tres letras).

3.4.2 En la clase SalidaPantalla

Otro nuevo aspecto del programa es que es necesario mostrar el ranking de puntuaciones en terminar el juego. Dado el planteamiento de la clase Ranking, esta tarea es muy simple y podría llevarse a cabo directamente en JocArena. Sin embargo, para ser coherentes con el hecho de que los métodos que muestran varias líneas de texto por pantalla, como el estado del luchador, están en esta clase, se será estricto en la aplicación de la modularidad tal y como se ha planteado en la aplicación originalmente.

Como antes, note que el cambio de la clase Bestiario no afecta en absoluto a esta clase, aunque la utiliza. También observa ahora cómo se comprueba la propagación de errores desde el método leerRanking, mirando si se ha evaluado null o no.

```
1 //Ahora hay que importar la nueva versión
2     importe juego.arena.ficheros.Bestiario;
3     //Nuevo método
4     /** Muestra por pantalla la lista de puntuaciones
5         */
6     public void mostrarRanking() {
7         Ranking rnk = new Ranking();
8         String s = rnk.leerRanquing();
9         if (s == null) {
10             System.out.println("¡Hay un error en los archivos de puntuaciones!");
11         } else {
12             System.out.println(s);
13         }
14     }
```

3.4.3 En la clase JuegoArena

En el apartado de anexos de los materiales dispone del código fuente completo con la solución final de este ejemplo.

En esta clase, sólo es necesario modificar el método inicio para que, el programa prevea las nuevas funcionalidades. Por un lado, ahora hay que garantizar que se inicializan los valores de los adversarios a partir de un archivo. Por otra parte, al finalizar la partida, es necesario que se compruebe si procede añadir una nueva máxima puntuación al archivo y realizar las operaciones correspondientes al teclado y la pantalla (entrar iniciales y mostrar ranking actual).

Aunque se ha intentado minimizar el número de cambios necesarios debido a la nueva clase Bestiario, es necesaria alguna modificación, ya que al menos hay que controlar si el archivo de adversarios existe antes de hacer nada. Por lo demás, el cambio de clase no afecta en nada más.

```
1 //Ahora hay que importar la nueva versión
2     importe juego.arena.ficheros.Bestiario;
3     public void inicio() {
4         //Mirar si hay archivo de adversarios
5         if (bestiario.existeArchivo() == false) {
6             System.out.println("¡No hay archivo de adversarios!");
7         } else {
8             //Código original
9             salida.mostrarBienvenida();
10            int[] jugador = bestiario.generarJugador();
```

```
11     int numCombat = 0;
12     boolean jugar = true;
13     while (jugar) {
14         numCombat++;
15         //Antes de cada combate se restaura el jugador
16         luchador.restaurar(jugador);
17         //Inicio de un combate
18         System.out.println("**** COMBAT" + numCombat);
19         System.out.print("Estado actual del jugador: ");
20         salida.mostrarLuchador(jugador);
21         System.out.println("*****");
22         //Se obtiene el adversario
23         int[] adversario = entrada.elegirAdversario(luchador.leerNivel(jugador)
24             ));
25         //Combate
26         combatir(jugador, adversario);
27         //Fi
28         jugar = fiCombat(jugador, adversario);
29         if (numCombat == MAX_COMBAT) {
30             System.out.println("Has sobrevivido a todos los combates. ¡¡¡Enhorabuena!!"
31                 );
32         }
33     }
34     System.out.print("Estado final del jugador: ");
35     salida.mostrarLuchador(jugador);
36     //Nuevo: comprobar si entra en el ranking (10 posiciones)
37     Ranking rnk = new Ranking();
38     int puntos = luchador.leerPuntos(jugador);
39     int pos = rnk.cercarRanking(puntos);
40     if (pos != -1) {
41         if (pos < 10) {
42             String iniciales = entrada.preguntarIniciales();
43             err = rnk.entrarPuntuacion(iniciales, puntos, pos);
44             if (err == -1) {
45                 System.out.println("Error accediendo al archivo de puntuaciones.");
46             }
47         }
48         salida.mostrarRanking();
49     } else {
50         System.out.println("Error accediendo al archivo de puntuaciones.");
51     }
52 }
```